Управление процессами машинного обучения (MLOps)

Лабораторный практикум

Рабочая редакция для 2026–2027 учебного года
Дата актуализации: 20 октября 2026 года
Направление подготовки: 09.03.02 «Информационные системы и технологии»
Профиль: «Машинное обучение и искусственный интеллект»
Семестр: 7

Назначение практикума

Практикум объединяет 16 лабораторных работ сквозного MLOps-проекта. Каждая команда развивает один репозиторий, а каждая работа добавляет проверяемый слой: данные, обучение, тестирование, поставку, эксплуатацию, governance или release. Приватная страница СКИФ не требуется для понимания задания.

Организация потока и команд

- поток — не менее 30 студентов;
- команда — от 2 до 5 человек;
- рекомендуемый вариант для 30 студентов — шесть команд по 4 и две команды по 3 человека;
- роли ротируются; один участник не остаётся единственным владельцем одной зоны более двух ЛР;
- общий результат команды не заменяет индивидуальную сдачу каждого студента.

Единый цикл выполнения лабораторной работы

1. Создать issue с критериями приёмки и назначить владельца.
2. Зафиксировать исходную identity: branch, commit, data/model/image version по контексту ЛР.
3. Выполнить изменение в короткоживущей ветке.
4. Запустить обязательные автоматические и ручные проверки.
5. Создать небольшой pull request и получить содержательный review другого участника.
6. Сохранить evidence: output, report, run URL, manifest, dashboard или иной артефакт.
7. Обновить индивидуальную evidence card и пройти защиту.
8. После принятия отметить неизменяемую командную версию tag или commit SHA.

Общие правила безопасности

Запрещено публиковать токены, пароли, персональные данные и закрытые учебные материалы. Секреты передаются через environment/secrets и не выводятся в лог. Опасные отказные сценарии выполняются на копии данных/spec или в локальной учебной среде. Плавающий image tag latest не используется для проверяемого release.

Индивидуальная evidence card

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	Номер, название, commit SHA или tag командной версии

Поле	Что указать
Роль	Роль в текущей ЛР и отличие от предыдущей
Задача	Собственная issue с критерием приёмки
Личный артефакт	PR, diff, review, test, run, report, dashboard, runbook или manifest
Вклад	Что лично создано, исправлено или проверено
Решение	Выбранный вариант, рассмотренная альтернатива и проверяемое основание
Риск	Возможный отказ и способ его обнаружения
Результат обучения	Что студент способен воспроизвести самостоятельно
Использование ИИ	Инструмент и самостоятельно выполненная проверка результата

Минимальный допуск к защите

1. Командная версия зафиксирована неизменяемой identity.
2. У студента есть собственная issue и содержательный личный артефакт.
3. Автор и reviewer изменения не совпадают.
4. Evidence card содержит рабочие ссылки и соответствует истории репозитория.
5. Обязательные проверки успешны либо исключение письменно принято преподавателем.
6. Студент способен выполнить диагностическую операцию без помощи команды.

Защита и повторная сдача

Студент за 2–4 минуты показывает личный артефакт в командной версии, объясняет входы, выходы, критерий корректности и риск, затем выполняет выбранную преподавателем диагностику. Оценка участника может отличаться от оценки команды. При повторной сдаче issue переоткрывается, исправление проходит отдельным PR, evidence card обновляется, а история первой попытки сохраняется.

Содержание

ЛР1. Репозиторий команды и воспроизводимая среда ЛР2. Контракт данных и предотвращение утечки ЛР3. Версионирование данных и воспроизводимые вычисления ЛР4. Эксперименты и базовая оценка модели ЛР5. Тестирование данных модели и pipeline ЛР6. Контейнеризация и управление средой ЛР7. Сервис предсказаний и контракт API ЛР8. Непрерывная интеграция и quality gates ЛР9. Оркестрация обучения и обработка отказов ЛР10. MLflow Registry и управляемый выпуск модели ЛР11. Наблюдаемость ML-сервиса ЛР12. Drift и отложенное качество модели ЛР13. Point-in-time признаки и Feature Store ЛР14. Документация, безопасность и ответственное применение ML ЛР15. Нагрузочный протокол, масштабирование и обзор LLMOps ЛР16. Интеграция, release manifest и защита проекта

Часть I. Репозиторий, данные и воспроизводимость

ЛР1. Репозиторий команды и воспроизводимая среда

Версия: 2026–2027 учебный год
Формат: командная работа по 2–5 человек с индивидуальной сдачей
Продолжение: результат ЛР1 используется во всех последующих лабораторных работах

1 Назначение работы

Команда должна подготовить единый GitHub-репозиторий учебного ML-проекта и доказать, что новый участник может получить код, создать изолированное окружение, установить зависимости

и выполнить базовые проверки только по README. В ЛР1 оценивается не сложность модели, а надежность командного процесса и воспроизводимость стартового состояния проекта.

2 Результаты обучения

После выполнения работы каждый студент должен уметь:

- объяснить различия между рабочей копией, локальным и удаленным репозиторием;
- создать ветку от актуального main, связать изменение с issue и открыть pull request;
- выполнить содержательное review изменения другого участника;
- восстановить Python-окружение по зафиксированным требованиям;
- объяснить назначение README, CONTRIBUTING, CODEOWNERS, ADR и CI workflow;
- показать, почему секреты, персональные данные, модели и локальное окружение не должны попадать в Git.

3 Командный результат

К завершению ЛР1 команда предоставляет:

1. fork тимлида от ветки своей команды в репозитории преподавателя и отдельные forks участников от назначенных веток тимлида;
2. ветку main тимлида и ветки участников, защищенные от прямых изменений организационными или техническими правилами;
3. не менее одного закрытого issue, связанного с принятым pull request;
4. review от участника, который не является автором последнего изменения;
5. README с проверенными командами установки, тестирования и запуска;
6. CONTRIBUTING с правилами веток, commit, pull request и review;
7. CODEOWNERS после замены учебных заполнителей реальными GitHub-логинами;
8. ADR 0001 с границами проекта, пользователем, метрикой успеха и основными рисками;
9. успешный CI run для принятого pull request;
10. неизменяемую контрольную точку lr1-v1 в виде Git tag или GitHub Release.

4 Подготовка

На рабочей машине требуются Git, GitHub-аккаунт, Python 3.12 и редактор кода. Docker для ЛР1 необязателен. Используйте стартовый проект 2026-2027/starter-project или выданный преподавателем GitHub template repository.

Проверьте инструменты:

```
git --version
python --version
python -m pip --version
```

Настройте автора commit на уровне репозитория или пользователя. Указывайте собственные данные, связанные с GitHub-аккаунтом:

```
git config user.name "Имя Фамилия"
git config user.email "адрес из настроек GitHub"
```

Не публикуйте токены, пароли, .env, приватные ключи и реальные персональные данные. Для переменных окружения храните только безопасный .env.example без значений секретов.

5 Роли команды в ЛР1

Роль	Ответственность	Индивидуальное подтверждение
Release lead	Формирует план ЛР1, контролирует связи issue–PR и создает tag lr1-v1	Закрытый issue, merge или release notes
Repository administrator	Настраивает доступ, правила main, шаблоны issue и pull request	Снимок или экспорт правил и проверяемый PR
Environment engineer	Проверяет Python-версию, установку зависимостей и чистый запуск	Изменение README, .python-version или сценария установки
Quality engineer	Настраивает и проверяет lint, тесты и CI	Workflow run, тест или исправление проверки
Documentation and risk reviewer	Оформляет ADR, CONTRIBUTING, CODEOWNERS и проверяет риски	Документ, содержательное review или запись решения

В команде из 2–4 человек роли совмещаются. В команде из пяти каждому назначается отдельная зона результата. Координация без технического, документального или проверочного артефакта не считается индивидуальным вкладом.

6 Рекомендуемая структура репозитория

```
project/
├── .github/
│   ├── ISSUE_TEMPLATE/
│   │   └── lab-task.yml
│   ├── workflows/
│   │   └── ci.yml
│   └── pull_request_template.md
├── data/
├── docs/
│   ├── adr/
│   │   └── 0001-project-boundaries.md
│   ├── evidence/
│   │   └── lr1-evidence-card.md
│   └── team-roles.md
├── src/
├── tests/
├── .gitignore
├── .python-version
├── CODEOWNERS
├── CONTRIBUTING.md
├── pyproject.toml
└── README.md
```

Данные, модели и артефакты экспериментов в ЛР1 не добавляются в Git, кроме небольшого разрешенного учебного примера. Политика управления данными будет уточнена в ЛР2.

7 Порядок выполнения

7.1 Создание команды, forks и веток

Преподаватель ведет ветку каждой команды, например team-01, в репозитории курса. Тимлид делает fork репозитория преподавателя и работает в своем fork: его main — интеграционная ветка команды. В fork тимлида создается отдельная ветка на каждого участника, например student-01, student-02.

Каждый участник делает fork **своей назначенной ветки тимлида**, а не независимый новый шаблон. В своем fork участник работает в main и открывает PR в соответствующую ветку тимлида. Так история личного вклада и review сохраняются проверяемыми.

1. Преподаватель фиксирует состав команды из 2–5 человек и ветку команды в репозитории курса.
2. Тимлид делает fork, создает ветки участников и сообщает каждому назначение.
3. Каждый участник делает fork назначенной ветки тимлида, записывает роль в docs/team-roles.md и настраивает remotes.
4. Участник создает PR из student-fork:main в teamlead-fork:student-XX.
5. Тимлид после review принимает PR участников, затем открывает и принимает PR из веток участников в teamlead-fork:main.
6. Для ЛР1 тимлид открывает PR из teamlead-fork:main в teacher-repository:team-XX. Ссылка на этот PR прикрепляется в СКИФ.

Пример настройки локальной копии участника:

```
git clone https://github.com/STUDENT_LOGIN/TEAM_REPOSITORY.git
cd TEAM_REPOSITORY
git remote add teamlead https://github.com/TEAMLEAD_LOGIN/TEAM_REPOSITORY.git
git remote add teacher https://github.com/Irina-64/T26_MLOps.git
git remote -v
git status
```

Для ЛР2–ЛР15 в СКИФ прикрепляется ссылка на main fork-а тимлида после интеграции личных PR. Для ЛР16 тимлид вновь открывает PR из своего main в ветку команды преподавателя и прикрепляет его ссылку в СКИФ. Прямая отправка изменений в репозиторий преподавателя не допускается.

7.2 Воспроизводимое окружение

Создайте окружение в каталоге проекта и установите пакет с зависимостями для разработки.

Windows PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
```

Linux или macOS:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
```

Выполните проверки:

```
python -m ruff check .
python -m pytest -q
```

Если команда использует другой менеджер окружений, она сохраняет эквивалентный lock-файл и добавляет проверенные команды в README. Локальное окружение .venv не коммитится.

7.3 Работа через issue и личный PR

Каждый участник создает или получает issue с наблюдаемым результатом и критериями приемки. Рабочая ветка создается от обновленного main **собственного fork-а**, затем изменения возвращаются PR в его назначенную ветку fork-а тимлида:

```
git switch main
git pull --ff-only
git switch -c docs/issue-12-reproducible-setup
```

После изменения проверьте diff и включайте в commit только относящиеся к задаче файлы:

```
git status
git diff
git add README.md .python-version
git commit -m "docs: document reproducible setup"
git push -u origin docs/issue-12-reproducible-setup
```

После push откройте PR STUDENT_LOGIN:docs/issue-12-reproducible-setup → TEAMLEAD_LOGIN:student-XX. Тимлид не подменяет личный PR прямым commit: сначала он одобряет PR участника, затем интегрирует принятые ветки в свой main. Не используйте git add ., если не проверили список файлов. В истории не должно быть секретов, локального окружения, кэшей, моделей и случайных наборов данных.

7.4 Pull request и review

Личный pull request должен ссылаться на issue, описывать результат и содержать способ проверки. Автор назначает reviewer из своей команды или тимлида. Reviewer:

- воспроизводит указанные команды либо проверяет связанный CI run;
- просматривает весь diff;
- оставляет конкретный комментарий, одобрение или запрос изменений;
- повторно проверяет изменение после исправлений.

Автор не утверждает собственный pull request. После новых commit требуется проверить, что одобрение относится к актуальному состоянию diff. Тимлид также не принимает собственный личный PR без независимого review.

7.5 Правила ветки main

Для main fork-а тимлида и для веток участников включите доступные в используемом плане GitHub правила:

- merge только через pull request;
- минимум одно одобрение другого участника;
- обязательное разрешение обсуждений;
- обязательная успешная проверка quality или эквивалентный CI job;
- запрет force push и удаления ветки;
- при возможности — одобрение последнего изменения участником, который его не отправлял.

Если учебный тариф или тип репозитория не позволяет применить правило технически, команда документирует эквивалентный процесс в CONTRIBUTING и демонстрирует его соблюдение по истории pull request. Ограничение платформы не отменяет review. До PR тимлида в ветку команды преподавателя в его main должны быть интегрированы и проверены личные PR всех участников.

7.6 Базовый CI

В стартовом проекте используется проверяемый workflow. На дату актуализации официальные репозитории действий рекомендуют актуальные major-версии actions/checkout@v7 и actions/setup-python@v7; перед новым потоком преподаватель сверяет их повторно.

```yaml
name: CI

on:
  push:
  pull_request:

permissions:
  contents: read
```

```yaml
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
          cache: pip
      - run: python -m pip install --upgrade pip
      - run: python -m pip install -e ".[dev]"
      - run: python -m ruff check .
      - run: python -m pytest -q
```

В ЛР1 не добавляйте секреты и публикацию образов. Разрешения workflow оставляйте минимальными.

7.7 ADR 0001 и границы проекта

В docs/adr/0001-project-boundaries.md зафиксируйте:

- учебную задачу и предполагаемого пользователя результата;
- входные данные без загрузки закрытого набора;
- основную метрику качества и эксплуатационный показатель;
- компоненты, которые войдут в итоговый проект;
- минимум три риска: данные, воспроизводимость и эксплуатация;
- решение, которое команда сознательно откладывает до следующих ЛР.

ADR описывает принятое решение и его последствия, а не пересказывает задание.

7.8 Контрольная точка

После принятия всех обязательных изменений:

```
git switch main
git pull --ff-only
git tag -a lr1-v1 -m "Laboratory 1 accepted baseline"
git push origin lr1-v1
```

Tag создается только на commit, прошедшем обязательный CI и review.

8 Сценарий приемки командного результата

Преподаватель или участник другой команды выполняет проверку в новом каталоге:

1. клонирует репозиторий по HTTPS;
2. создает чистое окружение по README;
3. устанавливает зависимости;
4. запускает Ruff и pytest;
5. находит закрытый issue, связанный pull request и review;
6. проверяет tag lr1-v1, ADR 0001 и отсутствие секретов;
7. сопоставляет правила main с CONTRIBUTING.

Командный результат не принимается, если успешный запуск зависит от незадокументированного локального файла, личного пути или устного пояснения автора.

9 Индивидуальная сдача участника команды

Каждый участник сдает ЛР1 отдельно и до защиты заполняет docs/evidence/lr1-evidence-card.md. Совпадающие карточки участников не принимаются.

Поле	Что указать для ЛР1
Студент	ФИО, учебная группа, команда и GitHub login
Контрольная версия	Ссылка на tag lr1-v1 и commit SHA
Роль	Роль в ЛР1 и зона ответственности
Задача	Собственная issue с критериями приемки
Личный артефакт	PR, содержательное review, workflow, README, CONTRIBUTING, CODEOWNERS, ADR или настройка правил
Вклад	Конкретное созданное, исправленное или проверенное изменение
Решение	Выбранный вариант и проверяемая причина выбора
Риск	Что могло пойти неверно и как это обнаруживается
Результат обучения	Что участник теперь может повторить самостоятельно
Использование ИИ	Инструмент, цель, способ проверки результата или отметка об отсутствии использования

9.1 Допуск к защите

Участник допускается, если:

1. командная версия зафиксирована tag lr1-v1;
2. личная issue содержит наблюдаемый результат;
3. есть содержательный личный артефакт в общей версии команды;
4. изменение прошло review другого участника либо сам участник выполнил содержательное review чужого изменения;
5. ссылки в evidence card открываются и соответствуют истории GitHub;
6. обязательный CI завершился успешно.

Количество commit само по себе не подтверждает вклад.

9.2 Индивидуальная защита

За 2–4 минуты участник:

1. показывает свой артефакт в командной версии;
2. объясняет входы, результат, критерий корректности и один риск;
3. выполняет одну операцию по выбору преподавателя: находит изменение через git log, объясняет diff, воспроизводит проверку, показывает связь issue–PR, интерпретирует CI log или объясняет правило main;
4. отвечает на один вопрос по соседнему компоненту, связанному с личным вкладом.

9.3 Стоп факторы

- чужой или отсутствующий личный артефакт;
- фиктивный commit либо искусственное дробление без самостоятельного результата;
- неработающая ссылка или несоответствие evidence card истории репозитория;
- секреты, персональные данные, локальное окружение или запрещенные данные в Git;
- прямой push в main в обход принятого процесса;
- неспособность объяснить собственное изменение.

9.4 Повторная сдача

Участник создает или переоткрывает issue с перечнем исправлений, вносит изменения отдельным pull request, получает новое review, обновляет evidence card и повторно защищает исправленную часть. История первой попытки не удаляется.

10 Оценивание

Критерий	Баллы	Проверка
Работоспособность	0–3	Репозиторий клонируется, окружение устанавливается, команды запуска выполняются
Воспроизводимость	0–2	Версии и зависимости зафиксированы, новый участник следует только README
Качество и безопасность	0–2	CI успешен, .gitignore корректен, секреты и лишние артефакты отсутствуют
Командный процесс	0–1	Issue, pull request и review прослеживаются, правила main соблюдены
Индивидуальная защита	0–2	Участник доказывает вклад и объясняет решение и риск

ЛР1 зачтена при результате не ниже 6 из 10 и отсутствии стоп-фактора. Командные пункты создают общий базовый балл, но итог каждого участника может отличаться по evidence card и защите.

11 Чек лист команды

- ☐ В репозитории работают ссылки из README.
- ☐ .venv, .env, кэши, модели и неразрешенные данные отсутствуют в Git.
- ☐ Все участники имеют собственные issue и проверяемые артефакты.
- ☐ Есть принятый pull request с review другого участника.
- ☐ CI выполняет Ruff и pytest с минимальными разрешениями.
- ☐ Настроены или документированы правила защиты main.
- ☐ Заполнены CONTRIBUTING, CODEOWNERS, ADR 0001 и таблица ролей.
- ☐ Evidence card каждого участника содержит уникальные рабочие ссылки.
- ☐ Tag lr1-v1 указывает на проверенный commit.

12 Вопросы для самопроверки

1. Чем git fetch отличается от git pull?
2. Почему ветку следует создавать от актуального main?
3. Что именно подтверждает pull request, а что — review?
4. Почему автор не должен быть единственным reviewer собственного изменения?
5. Какие файлы нельзя добавлять в Git и почему одного удаления недостаточно, если секрет уже попал в историю?
6. Чем фиксированная версия Python отличается от списка зависимостей проекта?
7. Зачем CI задавать минимальные permissions?
8. Как доказать, что tag lr1-v1 соответствует принятой версии?

13 Источники

- GitHub Docs: Protected branches и Pull request reviews.
- GitHub Docs Issue and pull request templates

- GitHub Actions: checkout и setup-python. Major-версии проверены 24 сентября 2026 года.

ЛР2. Контракт данных и предотвращение утечки

Версия: 2026-2027 учебный год
Формат: командная работа по 2-5 человек с индивидуальной сдачей
Предшествующая работа: ЛР1 Репозиторий команды и воспроизводимая среда
Продолжение: результаты ЛР2 используются в ЛР3 при построении версионируемого DVC-конвейера

1 Назначение работы

Команда должна превратить учебный набор данных из безымянного CSV-файла в управляемый артефакт. Для этого требуется зафиксировать происхождение и допустимое использование данных, создать понятную человеку карточку набора, описать машинно-проверяемый контракт и реализовать автоматическую проверку качества. Валидатор должен не только сообщать об ошибке, но и формировать воспроизводимый отчет с версией контракта и контрольной суммой входного файла.

В ЛР2 не требуется настраивать DVC, удаленное хранилище или полный обучающий pipeline. Эти задачи относятся к ЛР3. Здесь оценивается готовность данных к безопасному включению в такой pipeline.

2 Результаты обучения

После выполнения работы каждый студент должен уметь:

- объяснить различие между карточкой набора данных и машинным контрактом;
- проследить происхождение набора от источника до файла в проекте;
- определить обязательные поля, типы, допустимые значения, диапазоны и правила пропусков;
- реализовать автоматическую проверку схемы, полноты, уникальности и диапазонов;
- получить SHA-256 проверенного файла и связать его с отчетом валидации;
- зафиксировать момент прогноза и разделить train, validation и test строго по времени;
- объяснить, какие данные запрещено публиковать в Git;
- воспроизвести успешную и отрицательную проверку независимо от других участников команды.

3 Командный результат

К завершению ЛР2 команда предоставляет:

1. заполненную карточку набора docs/data-card.md;
2. версионируемый контракт data/contract.json;
3. валидатор src/mlops_flight_delay/validate_data.py с ненулевым кодом завершения при нарушении;
4. тесты контракта, включая не менее двух намеренно испорченных вариантов данных;
5. JSON-отчет успешной проверки с версией контракта, SHA-256, числом строк и результатами отдельных правил;
6. документированные границы хронологических train, validation и test и меры против утечки;
7. безопасный разрешенный образец данных вместо полного или закрытого набора;
8. issue и pull request с review участника, который не является автором изменения;
9. заполненную индивидуальную карточку docs/evidence/lr2-evidence-card.md для каждого участника;

10. принятую контрольную точку lr2-v1 в виде Git tag или GitHub Release после успешного CI.

4 Исходные данные и границы работы

В стартовом проекте используется файл data/flights_sample.csv из 24 строк. Он нужен для smoke-проверок; основная работа выполняется на подготовленном преподавателем фрагменте BTS объёмом 20–50 тысяч строк или на согласованном эквиваленте:

Поле	Роль	Смысл	Учебное ограничение
FlightDate	время события	дата рейса и ключ хронологического разбиения	формат YYYY-MM-DD, строки отсортированы по возрастанию
DepHour	признак	час отправления по местному времени	целое число от 0 до 23
IsWeekend	признак	признак выходного дня	только 0 или 1
ArrDelay	источник цели	задержка прибытия в минутах	целое число от -180 до 1440

Целевой класс формируется условием ArrDelay >= 15. Момент прогноза — за 60 минут до планового отправления. Сам ArrDelay и любые факты, появившиеся позже момента прогноза, нельзя подавать модели как признаки: это прямая утечка. Учебный файл синтетический, не содержит персональных данных и нужен для проверки инфраструктуры, а не для выводов о реальных рейсах.

Если преподаватель выдал другой набор, команда обязана адаптировать контракт и карточку под него. Нельзя механически сохранить диапазоны из примера. Для реальных данных укажите владельца, способ получения, дату или период, лицензию, ограничения использования, чувствительные поля и порядок удаления.

5 Роли команды в ЛР2

Роль	Ответственность	Индивидуальное подтверждение
Data steward	уточняет происхождение, лицензию, владельца и допустимое использование	разделы карточки данных и ссылка на источник или решение команды
Contract engineer	проектирует поля, типы, обязательность, диапазоны и версию контракта	diff data/contract.json и объяснение двух правил
Quality engineer	реализует валидатор и отрицательные тесты	код проверки, тест или диагностический отчет
Leakage reviewer	проверяет признаки, цель и способ разделения	запись решения, review или тест против утечки
Repository and documentation reviewer	контролирует Git-исключения, README, evidence cards и итоговый PR	содержательное review и проверяемое изменение документации

В команде из 2-4 человек роли совмещаются. В команде из пяти участники получают отдельные зоны ответственности. Название роли само по себе не подтверждает вклад: необходим собственный commit, review, тест, отчет или документируемое решение.

6 Подготовка репозитория

Начинайте от принятого результата ЛР1. Обновите локальную ветку и создайте отдельную ветку задачи:

```
git switch main
git pull --ff-only
git switch -c lab2/data-contract
python -m pip install -e ".[dev]"
```

Проверьте, что базовые тесты проходят до изменения данных:

```
python -m ruff check .
python -m pytest -q
```

Создайте issue с критериями готовности ЛР2. Назначьте владельцев артефактов и reviewer. Не добавляйте в issue секретные ссылки, токены доступа или фрагменты закрытых данных.

7 Шаг 1 Инвентаризация и происхождение данных

До написания кода ответьте на вопросы:

1. Кто создал или предоставил данные?
2. Когда и каким способом они получены?
3. На каком основании команда может их использовать и публиковать?
4. Содержат ли они персональные, коммерчески чувствительные или иные ограниченные сведения?
5. Какой файл разрешено хранить в Git?
6. Кто отвечает за обновление и удаление набора?
7. Как проверить, что используемый файл не изменился незаметно?

Для полного реального набора зафиксируйте внешнее расположение или идентификатор версии, но не помещайте закрытые данные в репозиторий. В Git можно оставить небольшой обезличенный образец, если это разрешено владельцем и лицензией. Добавьте каталоги data/raw/ и data/processed/ в .gitignore.

Получите контрольную сумму учебного файла:

```
python -c "from pathlib import Path; import hashlib; p=Path('data/flights_sample.csv'); print(hashlib.sha256(p.read_bytes()).hexdigest())"
```

Контрольная сумма не доказывает качество или законность данных. Она позволяет однозначно связать отчет с конкретными байтами файла.

8 Шаг 2 Карточка набора данных

Карточка docs/data-card.md объясняет набор человеку. Она должна содержать:

- название, назначение и запрещенные сценарии использования;
- происхождение, владельца, способ получения и период;
- лицензию или иное основание использования;
- состав полей и способ построения целевой переменной;
- размер, единицы измерения и известные пропуски;
- чувствительные данные и примененное обезличивание;
- ограничения репрезентативности и известные смещения;
- стратегию разделения и риск утечки;
- правила обновления, версионирования и удаления.

Карточка не должна повторять весь JSON-контракт. Ее задача - объяснить контекст и риски. Контракт, напротив, служит точным исполнимым источником требований.

9 Шаг 3 Машинный контракт данных

Контракт хранится в data/contract.json. Минимальная структура:

```
{
  "contract_version": "1.0.0",
  "dataset": "flight-delay-training-sample",
  "minimum_rows": 20,
```

```
  "allow_extra_columns": false,
  "duplicate_rows": "forbid",
  "columns": {
    "FlightDate": {
      "role": "event_time",
      "type": "date",
      "format": "%Y-%m-%d",
      "nullable": false
    },
    "DepHour": {
      "role": "feature",
      "type": "integer",
      "nullable": false,
      "minimum": 0,
      "maximum": 23
    },
    "IsWeekend": {
      "role": "feature",
      "type": "integer",
      "nullable": false,
      "allowed_values": [0, 1]
    }
  }
}
```

Каждое правило должно иметь предметный смысл. Например, диапазон часа от 0 до 23 следует из единицы измерения, а не из минимума и максимума текущего файла. Если правило основано только на наблюдаемой статистике, обозначьте его как предупреждение или обоснуйте запас.

Версию контракта меняйте осознанно:

- исправление описания без изменения проверки - patch;
- совместимое добавление необязательного поля - minor;
- удаление или переименование поля, изменение типа или смысла - major.

Это учебное правило версионирования должно быть зафиксировано командой в README или карточке данных.

10 Шаг 4 Автоматическая валидация

Валидатор стартового проекта запускается так:

```
python -m mlops_flight_delay.validate_data \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --report reports/validation/lr2.json
```

В PowerShell можно записать команду в одну строку. Успешная проверка должна завершаться кодом 0, неуспешная - ненулевым кодом. Отчет должен содержать не менее следующих полей:

Поле отчета	Назначение
contract_version	связывает результат с правилами проверки
dataset	идентифицирует логический набор
data_file	показывает проверенный путь
sha256	идентифицирует содержимое файла
rows и columns	фиксируют базовую форму набора
passed	дает итоговый результат для CI

Поле отчета	Назначение
checks	объясняет прохождение или отказ по каждому правилу

Минимальный набор автоматических проверок:

- наличие обязательных и отсутствие неожиданных полей;
- минимальное число строк;
- запрет пропусков в обязательных полях;
- преобразуемость в заявленный тип и целочисленность;
- допустимые диапазоны и категории;
- запрет полных дубликатов;
- ненулевой код завершения при любом нарушении.

Не ловите все исключения с последующим успешным завершением. Ошибка чтения файла, поврежденный контракт или нарушение данных должны быть видны пользователю и CI.

11 Шаг 5 Негативные сценарии

Положительный тест доказывает только то, что текущий пример проходит. Для приемки создайте во временном каталоге не менее двух испорченных вариантов:

1. значение DepHour=25, нарушающее верхнюю границу;
2. отсутствие ArrDelay или недопустимое значение IsWeekend=2.

Не изменяйте эталонный flights_sample.csv ради демонстрации. Тест должен построить временный файл, вызвать валидатор и проверить, что нужное правило стало неуспешным. Дополнительно рекомендуются сценарии с пропуском, дробным значением целочисленного поля, неизвестным столбцом, полным дубликатом и недостаточным числом строк.

Запустите проверки:

```
python -m pytest -q tests/test_data_contract.py
python -m ruff check src tests
```

В CI валидатор должен выполняться до обучения модели. Если данные не соответствуют контракту, дальнейшие этапы не запускаются.

12 Момент прогноза, разбиение и предотвращение утечки

Сначала зафиксируйте prediction timestamp: прогноз строится за 60 минут до планового отправления. Для каждого потенциального признака укажите event timestamp и available timestamp. Признак допускается только тогда, когда его значение гарантированно известно к prediction timestamp.

После сортировки по времени задайте три непересекающихся периода:

- train — самый ранний период, на нём обучаются преобразования и модель;
- validation — следующий период, на нём выбираются решение и пороги;
- test — самый поздний период, он используется один раз для итоговой независимой оценки.

В стартовом образце используется пропорция 60/20/20. Для данных BTS команда фиксирует календарные границы и количество строк в каждой части. Перемешивание и случайная стратификация запрещены. Статистики нормализации, словари категорий и заполнение пропусков обучаются только на train. ArrDelay используется только для построения отложенной цели и не входит в FEATURES.

В validation report добавьте границы периодов, число строк, долю положительного класса и проверку max(train_time) < min(validation_time) <= max(validation_time) < min(test_time).

13 Git и безопасность данных

Запрещено добавлять в репозиторий:

- полные закрытые или лицензионно ограниченные наборы;
- персональные данные и их необезличенные фрагменты;
- секреты, токены, строки подключения и подписанные URL;
- выгрузки с неизвестным происхождением;
- временные обработанные копии, если они воспроизводимы;
- файлы, превышающие правила репозитория и не предназначенные для Git.

Если чувствительный файл уже попал в commit, одного удаления в новом commit недостаточно: значение остается в истории. Остановите публикацию, отзовите скомпрометированные секреты, сообщите преподавателю или владельцу и примените согласованную процедуру очистки истории.

14 Pull request и контрольная точка

Pull request ЛР2 должен включать контракт, карточку данных, валидатор, тесты, README и индивидуальные evidence cards. В описании укажите:

- источник и разрешенное использование набора;
- версию контракта;
- команду успешной проверки;
- два отрицательных сценария;
- SHA-256 принятого образца;
- выбранную стратегию разделения;
- известные ограничения и отложенные задачи для ЛР3.

Reviewer сверяет правила с предметным смыслом, запускает проверки локально и оставляет содержательный комментарий. После успешного CI и merge создайте tag lr2-v1. Отчет валидации должен быть воспроизводим по tag, даже если команда не хранит генерируемый JSON в Git.

15 Правила индивидуальной сдачи

Командный pull request необходим, но не заменяет индивидуальную защиту. Каждый участник копирует шаблон docs/evidence/lr2-evidence-card.md, называет файл по правилам преподавателя и заполняет его самостоятельно.

15.1 Что предоставляет каждый участник

Элемент	Обязательное содержание
Идентификация	ФИО, группа, команда, GitHub-логин и роль
Собственный вклад	2-4 изменения со ссылками на issue, PR, commit и строки diff
Проверка коллеги	ссылка на review, найденная проблема или проверенный критерий
Воспроизведение	команда запуска, SHA-256, границы периодов и результат успешной проверки
Отрицательный сценарий	самостоятельно внесенное нарушение и название сработавшей проверки
Обоснование	правило контракта, риск утечки и ограничение набора

15.2 Демонстрация на защите

Каждый студент без подсказки другого участника должен:

1. получить актуальный tag или commit команды;

2. запустить валидатор на корректном образце;
3. назвать версию контракта и показать SHA-256 в отчете;
4. создать во временной копии одно нарушение и получить ненулевой код завершения;
5. объяснить собственный вклад по diff;
6. объяснить, почему выбранный преподавателем признак доступен или недоступен за 60 минут до отправления.

Преподаватель вправе выбрать другой столбец или нарушение, чем указано в карточке. Демонстрация проводится на собственном окружении студента или в чистом клоне по указанию преподавателя.

15.3 Что не засчитывается как индивидуальный вклад

- только участие в созвоне или распределении задач;
- одинаковый текст evidence card у нескольких студентов;
- ссылка на общий PR без указания собственных изменений;
- commit, полностью созданный другим участником;
- формальное одобрение PR без проверки или комментария;
- демонстрация, которую может выполнить только один член команды;
- исправление данных вручную без автоматического правила и теста.

16 Критерии оценивания

Критерий	Баллы	Признаки полного выполнения
Функциональность	0-3	контракт и валидатор проверяют схему, типы, пропуски, диапазоны, категории и дубликаты; код завершения корректен
Воспроизводимость	0-2	команды документированы, отчет содержит версию и SHA-256, tag lr2-v1 воспроизводится
Качество и безопасность	0-2	есть карточка, происхождение, ограничения, стратегия против утечки и безопасная политика Git
Командный процесс	0-1	issue, разделение ответственности, PR и содержательное review прослеживаются
Индивидуальная защита	0-2	студент доказывает вклад, выполняет положительную и отрицательную проверку и объясняет решения

Максимум: 10 баллов. Рекомендуемый порог зачета: 6 баллов при отсутствии стоп-факторов.

Стоп-факторы: отсутствие собственного вклада или защиты; публикация закрытых данных или секрета; валидатор всегда возвращает успех; контракт не соответствует фактическим полям; целевая переменная используется как признак; результат невозможно воспроизвести по принятому commit.

17 Контрольный чек-лист команды

- ☐ происхождение, права использования и владелец данных зафиксированы;
- ☐ в Git находится только разрешенный обезличенный образец;
- ☐ карточка данных описывает назначение, состав, ограничения и риски;
- ☐ контракт содержит версию и проверяемые правила для каждого поля;
- ☐ валидатор возвращает код 0 только для корректного набора;
- ☐ JSON-отчет содержит SHA-256 и результаты отдельных проверок;
- ☐ два намеренных нарушения обнаруживаются автоматическими тестами;
- ☐ стратегия разделения и меры против утечки объяснены;

- ☐ CI выполняет lint, тесты и проверку данных до обучения;
- ☐ каждый участник подготовил отдельную evidence card;
- ☐ PR прошел независимое review и создан tag lr2-v1.

18 Вопросы для защиты

1. Почему карточка данных не заменяет контракт и наоборот?
2. Почему диапазон нельзя бездумно взять из минимума и максимума текущей выборки?
3. Что изменится в контракте при добавлении необязательного поля?
4. Как SHA-256 помогает воспроизводимости и чего он не гарантирует?
5. Почему ArrDelay нельзя использовать в качестве признака?
6. Когда случайное разделение хуже временного или группового?
7. Почему удаление секрета новым commit не устраняет утечку из истории Git?
8. Как валидатор должен вести себя при поврежденном JSON-контракте?
9. Чем предупреждение качества отличается от блокирующего правила?
10. Какие доказательства показывают именно ваш индивидуальный вклад?

19 Рекомендуемая структура результата

```
project/
├── data/
│   ├── contract.json
│   └── flights_sample.csv
├── docs/
│   ├── data-card.md
│   └── evidence/
│       └── lr2-<github-login>.md
├── reports/
│   └── validation/
├── src/mlops_flight_delay/
│   └── validate_data.py
├── tests/
│   └── test_data_contract.py
├── .gitignore
└── README.md
```

Каталог с отчетами может быть исключен из Git, если отчет надежно создается командой и сохраняется как артефакт CI. Решение должно быть единым для команды и описано в README.

20 Источники и дальнейшая работа

- документация Python по json, hashlib и кодам завершения;
- документация pandas по чтению CSV, пропускам, дубликатам и преобразованию типов;
- документация GitHub по защите чувствительных данных и review pull request;
- Gebru et al. Datasheets for Datasets - подход к документированию происхождения, состава, использования и ограничений набора;
- технологический справочник курса Технологический стек MLOps 2026-2027.md.

В ЛР3 контракт и валидатор становятся входным шлюзом DVC-конвейера: команда версионирует происхождение данных и зависимости этапов, не ослабляя принятые в ЛР2 правила.

ЛР3. Версионирование данных и воспроизводимые вычисления

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Предшествующая работа: ЛР2 «Контракт данных и предотвращение утечки»

1 Цель

Связать версию проверенных данных с Git-состоянием проекта, воспроизвести две версии набора через DVC и доказать восстановление по контрольной сумме. В этой работе DVC не заменяет контракт данных: после каждого восстановления набор снова проходит валидатор ЛР2.

2 Результаты обучения

Каждый студент должен уметь:

- объяснить, почему крупные данные не хранятся непосредственно в Git;
- различать Git commit, .dvc-метаданные, DVC cache и remote;
- добавить разрешённый набор под контроль DVC;
- настроить локальное remote-хранилище без секретов в репозитории;
- получить и восстановить две версии данных;
- проверить SHA-256 и контракт после dvc pull или dvc checkout;
- объяснить, какие файлы нужно передавать коллеге для воспроизведения.

3 Командный результат

Команда предоставляет:

1. установленный и инициализированный DVC;
2. файл метаданных набора и корректный .gitignore;
3. локальное DVC remote-хранилище, путь к которому не зависит от персонального каталога участника;
4. два Git commit, соответствующие версиям данных v1 и v2;
5. успешный validation report для каждой версии;
6. протокол восстановления с командами, commit SHA, DVC hash и SHA-256 файла;
7. тест, показывающий отказ при повреждении или несогласованной версии данных;
8. issue, pull request и review другого участника;
9. индивидуальную evidence card каждого студента.

4 Границы и безопасность

Для демонстрации используется разрешённый учебный фрагмент. Полные закрытые данные, персональные сведения, токены и ключи облачных хранилищ запрещено добавлять в Git и общий DVC remote. Локальный каталог remote должен находиться вне рабочего дерева, например в каталоге, выданном преподавателем. В Git сохраняется конфигурация remote без секретов; персональные пути при необходимости помещаются в локальную конфигурацию DVC.

5 Роли

Роль	Зона ответственности	Индивидуальное доказательство
Data version owner	формирует осмысленные v1 и v2	diff данных, описание причины изменения
DVC engineer	инициализация, cache и remote	команды и diff конфигурации
Reproducibility engineer	чистое восстановление	протокол с hash и контрольными суммами

Роль	Зона ответственности	Индивидуальное доказательство
Quality reviewer	повторный запуск контракта	два validation report и отрицательный тест
Documentation reviewer	README, PR и evidence	содержательное review и исправление

При меньшем составе роли совмещаются, но у каждого остаётся собственный проверяемый вклад.

6 Подготовка

Создайте ветку и убедитесь, что ЛР2 принята:

```
git switch main
git pull --ff-only
git switch -c lab3/dvc-versioning
python -m pip install "dvc>=3,<4"
python -m pytest -q
```

Зафиксируйте версии:

```
python --version
dvc version
git rev-parse HEAD
```

7 Инициализация DVC

В корне репозитория выполните:

```
dvc init
git status --short
git add .dvc .dvcignore
git commit -m "chore: initialize DVC"
```

До commit изучите созданные файлы. Объясните, почему .dvc/cache не должен попадать в Git и почему служебные файлы DVC, напротив, должны версионироваться.

8 Версия данных v1

Сначала запустите контракт ЛР2 и сохраните отчёт:

```
python -m mlops_flight_delay.validate_data --data data/flights_sample.csv --contract data/contract.json
--report reports/validation/lr3-v1.json
dvc add data/flights_sample.csv
git add data/flights_sample.csv.dvc data/.gitignore reports/validation/lr3-v1.json
git commit -m "data: register flight sample v1"
```

Если политика репозитория исключает генерируемые reports, сохраните отчёт как CI artifact и укажите URI в протоколе. Запишите Git commit, DVC hash из .dvc-файла и SHA-256 исходного CSV.

9 Локальное remote-хранилище

Пример для каталога, созданного преподавателем рядом с репозиторием:

```
dvc remote add -d course-local ../dvc-storage
dvc push
dvc remote list
```

Не используйте абсолютный персональный путь в общей конфигурации. Если это неизбежно, задайте его через --local и документируйте, что коллеге нужно настроить собственный путь.

Проверьте восстановимость: удалите только рабочую копию отслеживаемого CSV предусмотренной DVC-командой или переместите её во временный каталог, затем выполните dvc pull. Не удаляйте каталог проекта и remote.

10 Версия данных v2

Создайте осмысленное изменение: добавьте разрешённые строки более позднего периода или исправьте документированную ошибку. Сохраняйте сортировку по FlightDate и не меняйте test-период задним числом без отдельного решения.

```
python -m mlops_flight_delay.validate_data --data data/flights_sample.csv --contract data/contract.json
--report reports/validation/lr3-v2.json
dvc add data/flights_sample.csv
dvc push
git add data/flights_sample.csv.dvc reports/validation/lr3-v2.json
git commit -m "data: update flight sample to v2"
```

В описании commit укажите причину, источник новых строк, изменение периода и влияние на границы train/validation/test.

11 Восстановление двух версий

Протокол выполняется из чистого клона или после безопасного сохранения незакоммиченных изменений.

Для каждой версии:

```
git switch --detach <commit-v1-or-v2>
dvc pull
python -m mlops_flight_delay.validate_data --data data/flights_sample.csv --contract data/contract.json
--report reports/validation/restored.json
```

Сравните полученный SHA-256 с записанным значением. После проверки вернитесь в рабочую ветку. Не применяйте git reset --hard как способ очистки: он может уничтожить незакоммиченные изменения.

12 Отказные сценарии

Проверьте минимум два сценария:

1. Рабочий CSV изменён вручную без dvc add: dvc status показывает несогласованность.
2. Восстановленная версия нарушает контракт или порядок дат: валидатор возвращает ненулевой код.

Дополнительно можно временно сделать remote недоступным и описать понятное сообщение DVC. После эксперимента восстановите конфигурацию.

13 Протокол воспроизводимости

Создайте docs/reproducibility/lr3.md с таблицей:

Версия	Git commit	DVC hash	SHA-256 CSV	Период	Строк	Контракт	Результат
v1	...	...	...	...	...	2.0.0	passed
v2	...	...	...	...	...	2.0.0	passed

Ни один hash не заменяет остальные: Git идентифицирует проект, DVC — объект данных, SHA-256 отчёта связывает конкретный файл с проверкой.

14 Индивидуальная сдача

Каждый участник заполняет отдельную карточку:

```
## Индивидуальная карточка ЛР3

- ФИО, группа, команда, GitHub-логин:
- роль:
- issue, pull request, commit:
- мои 2–4 изменения со ссылками на diff:
- review работы другого участника и найденная проблема:
```

- команда восстановления выбранной версии:
- Git commit, DVC hash и SHA-256:
- результат валидатора:
- отказной сценарий и объяснение:
- чем Git отличается от DVC cache и remote:
- вопрос преподавателя и ответ:

На защите преподаватель назначает v1 или v2. Студент самостоятельно восстанавливает её, запускает валидатор, показывает три идентификатора и объясняет собственный diff. Для команд из пяти человек проверяются разные действия или версии.

15 Критерии оценивания

Критерий	Баллы	Полное выполнение
DVC и структура репозитория	2	инициализация, метаданные и ignore корректны
Две версии данных	2	осмысленные v1/v2 связаны с двумя Git commit
Remote и восстановление	2	обе версии получаются из remote в чистом состоянии
Контракт и отказ	2	отчёты успешны, отрицательные сценарии воспроизводимы
Командный процесс	1	issue, PR и содержательное review
Индивидуальная защита	1	собственное восстановление, вклад и объяснение

Критические условия: отсутствие индивидуальной демонстрации, публикация запрещённых данных или невозможность восстановить назначенную версию означают, что работа не принята до исправления.

16 Чек-лист приёмки

- ☐ Команда состоит из 2–5 человек.
- ☐ В Git нет полного набора, DVC cache и секретов.
- ☐ Зафиксированы два Git commit и два DVC hash.
- ☐ Обе версии находятся в remote и восстанавливаются.
- ☐ Для обеих версий проходит контракт 2.0.0.
- ☐ SHA-256 после восстановления совпадает с протоколом.
- ☐ Есть минимум два отказных сценария.
- ☐ PR прошёл review другого участника.
- ☐ У каждого студента есть собственная evidence card и индивидуальная демонстрация.

17 Связь с ЛР4

В ЛР4 каждая MLflow run должна ссылаться на Git commit и версию данных из протокола ЛР3. Сравнение экспериментов без этой связи не считается воспроизводимым.

ЛР4. Эксперименты и базовая оценка модели

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Предшествующая работа: ЛР3 «Версионирование данных и воспроизводимые вычисления»

1 Цель

Сделать обучение модели наблюдаемым и сравнимым: зарегистрировать не менее двух MLflow runs, связать каждый запуск с Git commit и версией данных, сохранить параметры, метрики, входной набор и артефакт pipeline, а затем обосновать выбор базовой конфигурации только по validation-периоду.

2 Результаты обучения

Каждый студент должен уметь:

- различать experiment, run, parameter, metric, tag, input и artifact;
- запускать локальный MLflow Tracking Server или согласованный преподавателем файловый backend;
- логировать параметры модели и временного разбиения;
- связать run с Git commit, DVC hash, SHA-256 и версией контракта;
- регистрировать training и validation datasets как входы;
- сравнивать не менее двух запусков по одной и той же методике;
- объяснять различие validation и test и не выбирать модель по test;
- загрузить сохранённый pipeline-артефакт и повторить предсказание.

3 Командный результат

Команда предоставляет:

1. experiment с названием по шаблону преподавателя;
2. не менее двух завершённых MLflow runs с различающимся гиперпараметром или алгоритмом;
3. для каждого run: параметры, validation-метрики, dataset inputs, теги версий и модельный артефакт;
4. единый отчёт сравнения reports/experiments/lr4-comparison.md;
5. JSON-артефакт с границами train, validation и test и числами строк;
6. проверку загрузки модели из артефакта и повторного предсказания;
7. issue, pull request и содержательное review;
8. отдельную evidence card каждого участника;
9. tag lr4-v1 после успешных проверок.

4 Ограничение оценки

В ЛР4 модель выбирается только по validation-периоду. Test сохраняется закрытым для итоговой проверки и не используется для перебора параметров. Основная метрика учебного бинарного классификатора — ROC AUC на validation; дополнительно фиксируются precision, recall, F1 и доля положительного класса. Если в части данных есть только один класс, run завершается понятной ошибкой, а не фиктивной метрикой.

Небольшой синтетический набор стартового проекта предназначен для smoke-проверки. Содержательное сравнение выполняется на подготовленном преподавателем временном фрагменте данных.

5 Роли команды

Роль	Ответственность	Индивидуальное доказательство
Experiment owner	схема experiment и обязательных tags	run и описание соглашения
Model engineer	две конфигурации обучения	diff параметров и код pipeline
Evaluation engineer	метрики и границы частей	тест метрик и comparison report
Reproducibility reviewer	Git/DVC/data linkage	проверка tags и восстановление run

Роль	Ответственность	Индивидуальное доказательство
Quality and documentation reviewer	PR, загрузка модели, evidence	содержательное review и smoke test

При меньшем составе роли совмещаются. Роль должна отличаться от двух предыдущих ЛР, если это возможно.

6 Подготовка окружения

Создайте ветку и установите дополнительный набор зависимостей:

```
git switch main
git pull --ff-only
git switch -c lab4/mlflow-tracking
python -m pip install -e ".[dev,tracking]"
python -m pytest -q
```

Зафиксируйте исходное состояние:

```
git rev-parse HEAD
dvc status
python -m mlops_flight_delay.validate_data --data data/flights_sample.csv --contract data/contract.json
```

Рабочее дерево перед запуском должно быть чистым либо run получает tag working_tree_dirty=true с объяснением. Для принимаемой версии рекомендуется чистое состояние.

7 Локальный Tracking Server

Для командной работы используйте локальную SQLite-базу и каталог артефактов внутри разрешённого учебного окружения:

```
mlflow server --host 127.0.0.1 --port 5000
```

Во втором терминале задайте tracking URI:

```
$env:MLFLOW_TRACKING_URI="http://127.0.0.1:5000"
```

В Linux или macOS используйте export. Не публикуйте MLflow server во внешнюю сеть без аутентификации, ограничения хостов и согласования с преподавателем. Папки mlruns, mlartifacts и локальная SQLite-база не добавляются в Git.

8 Соглашение об experiment и run

Название experiment: mlops-<group>-<team>-flight-delay. Run name содержит короткую конфигурацию, например rf-depth-3.

Обязательные tags:

Tag	Содержание
git.commit	полный Git commit SHA
git.dirty	true или false
data.dvc_hash	hash версии данных из ЛР3
data.sha256	SHA-256 CSV
data.contract_version	версия контракта
split.policy	chronological-60-20-20 или утверждённые даты
team.code	код команды без персональных данных
validation.status	candidate, rejected или invalid

Не используйте tag Production или stage как решение о выпуске. В ЛР4 Registry ещё не применяется; управляемый выпуск относится к ЛР10.

9 Что логировать

Параметры должны включать алгоритм, n_estimators, max_depth, random_state, порог целевого класса и границы частей. Метрики должны иметь явный контекст, например validation_roc_auc, а не просто score.

Для dataset inputs используйте API MLflow для pandas-наборов:

```
training_input = mlflow.data.from_pandas(
    train_frame,
    source="data/flights_sample.csv",
    name="flight-delay-train",
    targets="target",
)
mlflow.log_input(training_input, context="training")
```

Не логируйте полный закрытый набор как artifact. Dataset input должен сохранять разрешённые метаданные, digest, схему и источник. Политика курса и лицензия данных имеют приоритет над удобством интерфейса.

10 Два обязательных запуска

Выполните две конфигурации при неизменных данных и границах:

```
python -m mlops_flight_delay.train_mlflow --data data/flights_sample.csv --contract data/contract.json --n-estimators 80 --max-depth 3 --experiment mlops-<group>-<team>-flight-delay
python -m mlops_flight_delay.train_mlflow --data data/flights_sample.csv --contract data/contract.json --n-estimators 160 --max-depth 6 --experiment mlops-<group>-<team>-flight-delay
```

Изменение сразу нескольких факторов допустимо только при явно сформулированной гипотезе. Для учебного сравнения лучше менять один параметр, чтобы вывод был интерпретируем.

11 Артефакты run

Каждый run сохраняет:

- сериализованный pipeline или model bundle;
- split_manifest.json с временными границами;
- metrics.json с validation-метриками;
- validation report ЛР2;
- краткий run_summary.md с гипотезой и выводом;
- окружение или список зафиксированных зависимостей.

После завершения загрузите модельный артефакт через MLflow API или скачайте artifact в чистый временный каталог. Выполните предсказание на разрешённой строке validation и сравните результат с записанным smoke test.

12 Отчёт сравнения

В lr4-comparison.md включите таблицу:

Run ID	Конфигурация	Версия данных	Validation ROC AUC	Precision	Recall	F1	Решение
...	RF depth 3	...	...	...	...	...	candidate/rejected
...	RF depth 6	...	...	...	...	...	candidate/rejected

После таблицы сформулируйте гипотезу, наблюдение, ограничение выборки и решение. Разница метрик на маленьком наборе не доказывает превосходство модели. Если значения одинаковы, это тоже результат, требующий объяснения.

13 Автоматические проверки

Добавьте тесты, которые подтверждают:

1. хронологическое разбиение и отсутствие пересечения частей;
2. наличие обоих классов перед вычислением ROC AUC;
3. сохранение обязательных полей model bundle;
4. загрузку артефакта и форму предсказания;
5. отказ при неверном контракте или невозможной метрике.

MLflow server не должен быть обязательным для всех unit-тестов. Отделите чистые функции обучения и оценки от интеграционного теста tracking.

14 Индивидуальная сдача участника команды

Командный результат: tag lr4-v1, два run ID и отчёт сравнения.

Каждый участник до защиты размещает индивидуальную evidence card. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Роль	Роль в ЛР4 и отличие от роли в ЛР3
Задача	issue с личным критерием приёмки
Личный артефакт	PR, commit, review, тест, run ID или отчёт
Вклад	конкретные созданные или проверенные элементы
Инженерное решение	выбранный параметр, метрика или способ логирования и альтернатива
Риск	утечка test, несвязанная версия данных, неполный run или закрытые данные
Воспроизведение	run ID, Git SHA, DVC hash и команда загрузки артефакта
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: командный tag существует, личная issue закрыта проверяемым результатом, PR прошёл review другого участника, оба runs завершены, обязательные tags и artifacts доступны.

15 Порядок индивидуальной защиты

За 3–4 минуты студент:

1. открывает назначенный преподавателем run ID;
2. показывает связь с Git commit и DVC hash;
3. объясняет один параметр, одну метрику и один artifact;
4. загружает модельный артефакт или запускает smoke test;
5. сравнивает run с альтернативной конфигурацией;
6. объясняет, почему test не использовался для выбора.

Преподаватель может попросить найти run по tag, изменить один параметр, показать неуспешный run или объяснить расхождение метрики.

16 Повторная сдача и стоп-факторы

При повторной сдаче создаётся или переоткрывается issue, изменения вносятся новым PR, создаётся новый run, а история неуспешной попытки сохраняется.

Стоп-факторы:

- нет личного артефакта или студент не может объяснить заявленный run;

- run не связан с Git и данными;
- модель выбрана по test-периоду;
- закрытые данные или секреты опубликованы как artifacts;
- метрика скопирована вручную и не воспроизводится;
- один run выдан за сравнение двух конфигураций.

17 Критерии оценивания

Критерий	Баллы	Полное выполнение
Структура experiment и runs	2	два сопоставимых завершённых запуска
Прослеживаемость	2	Git, DVC, SHA-256, контракт и split связаны tags
Метрики и входы	2	корректные validation-метрики и dataset inputs
Артефакты	1	модель, split manifest, отчёт и повторная загрузка
Командный процесс	1	issue, PR и содержательное review
Индивидуальная защита	2	личный вклад, воспроизведение и объяснение риска

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

18 Чек-лист команды

- ☐ Состав команды 2–5 человек, роли ротированы.
- ☐ Данные и контракт соответствуют принятой ЛР3.
- ☐ Созданы два run с различающимися конфигурациями.
- ☐ Каждый run связан с Git commit и DVC hash.
- ☐ Логируются параметры, validation-метрики и dataset inputs.
- ☐ Test не используется для выбора.
- ☐ Модельный артефакт загружается и выполняет предсказание.
- ☐ Отчёт содержит сравнение и ограничение выводов.
- ☐ PR прошёл независимое review.
- ☐ Каждый участник подготовил evidence card и индивидуальную демонстрацию.

19 Связь с ЛР5

В ЛР5 зафиксированные функции данных, обучения и оценки будут покрыты систематическими тестами и fixtures. Выбранный run остаётся базовой линией, но не переводится в Registry до управляемого выпуска в ЛР10.

20 Официальные источники

- MLflow Tracking: https://mlflow.org/docs/latest/ml/tracking/
- MLflow Tracking Quickstart: https://mlflow.org/docs/latest/ml/getting-started/quickstart/
- MLflow Dataset Tracking: https://mlflow.org/docs/latest/dataset/

ЛР5. Тестирование данных модели и pipeline

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Предшествующая работа: ЛР4 «Эксперименты и базовая оценка модели»

1 Цель

Построить проверяемую пирамиду тестов для сквозного ML-pipeline: отделить быстрые unit-тесты от контрактных и интеграционных проверок, подготовить fixtures корректных и

испорченных данных, получить воспроизводимые pass/fail отчёты и доказать, что тест контролирует конкретный риск.

2 Результаты обучения

Каждый студент должен уметь:

- отличать тест функции, контракта, модели, API и интеграции;
- использовать pytest fixtures, tmp_path, параметризацию и pytest.raises;
- создавать минимальный набор данных для одного сценария;
- проверять отказ при неверной схеме, утечке, одном классе и некорректной вероятности;
- тестировать инварианты model bundle и форму предсказания;
- формировать JUnit XML и текстовый отчёт;
- объяснять, какой риск покрывает тест и почему тест может дать ложную уверенность;
- диагностировать намеренно внесённый дефект независимо от команды.

3 Командный результат

Команда предоставляет:

1. структуру тестов по слоям data, evaluation, model, API и pipeline;
2. общие fixtures в tests/conftest.py без зависимости от порядка тестов;
3. не менее трёх параметризованных негативных сценариев данных;
4. тест метрик с обоими классами и отказом при одном классе;
5. тест model bundle и детерминированного обучения;
6. интеграционный smoke test validate -> train -> load -> predict;
7. положительный отчёт reports/tests/lr5-pass.xml;
8. сохранённый фрагмент отрицательного отчёта с именем упавшего теста и причиной;
9. таблицу «риск — тест — ожидаемый отказ»;
10. issue, pull request, review и индивидуальные evidence cards;
11. tag lr5-v1 после успешной приёмки.

4 Пирамида тестов курса

Слой	Что проверяет	Пример	Частота
Unit	чистую функцию	вычисление metrics и validation правила	каждый commit
Contract	границу компонента	поля данных или API response	каждый pull request
Model	инварианты артефакта	признаки, версия, диапазон вероятности	каждый pull request
Integration	связку компонентов	validate, train, load, predict	каждый pull request
End-to-end	пользовательский путь	сервис в контейнере	перед release

ЛР5 концентрируется на первых четырёх слоях. Полный контейнерный end-to-end будет добавлен после ЛР6 и ЛР7.

5 Роли

Роль	Зона ответственности	Индивидуальное доказательство
Data test engineer	fixtures и негативные данные	параметризованный тест и объяснение правил
Model test engineer	bundle и предсказания	тест инварианта модели

Роль	Зона ответственности	Индивидуальное доказательство
Evaluation test engineer	metrics и пороги	тест обоих классов и исключения
Integration engineer	сквозной smoke test	отчёт и диагностика слоя отказа
Quality reviewer	структура, PR и test report	содержательное review и defect injection

Для команды 2–4 роли совмещаются. Один студент не остаётся единственным владельцем тестов более двух последовательных ЛР.

6 Подготовка

```
git switch main
git pull --ff-only
git switch -c lab5/testing-pipeline
python -m pip install -e ".[dev]"
python -m pytest --collect-only -q
```

Сначала сохраните исходный успешный отчёт. Если базовые тесты уже падают, ЛР5 начинается с отдельной issue по регрессии; нельзя скрывать исходный дефект добавлением skip.

7 Требования к тестам

Хороший тест имеет явное имя, минимальные входы, одну основную причину отказа и диагностическое сообщение. Он не зависит от сети, MLflow server, порядка выполнения и файлов, оставленных предыдущим тестом.

Запрещены проверки вида assert True, тесты без assertion, ловля любого исключения без проверки типа и тесты, которые изменяют эталонный CSV. Каждый временный artifact создаётся через tmp_path.

8 Общие fixtures

Создайте tests/conftest.py:

```
from pathlib import Path

import pandas as pd
import pytest


@pytest.fixture
def valid_frame() -> pd.DataFrame:
    return pd.read_csv(Path(__file__).parents[1] / "data" / "flights_sample.csv")


@pytest.fixture
def csv_factory(tmp_path: Path):
    def write(frame: pd.DataFrame, name: str = "input.csv") -> Path:
        path = tmp_path / name
        frame.to_csv(path, index=False)
        return path
    return write
```

Fixture возвращает новое состояние для теста. Не изменяйте объект session scope, если мутация может повлиять на другие сценарии.

9 Параметризованные тесты данных

Используйте @pytest.mark.parametrize для нескольких независимых нарушений:

```
@pytest.mark.parametrize(
    ("column", "value", "check_name"),
```

```
    [
        ("DepHour", 25, "DepHour.maximum"),
        ("IsWeekend", 2, "IsWeekend.allowed_values"),
        ("FlightDate", "bad-date", "FlightDate.date"),
    ],
)
def test_invalid_values_are_rejected(valid_frame, csv_factory, column, value, check_name):
    frame = valid_frame.copy()
    frame.loc[0, column] = value
    result = validate_dataset(csv_factory(frame), CONTRACT)
    assert result["passed"] is False
    assert any(check["name"] == check_name and not check["passed"] for check in result["checks"])
```

Идентификатор параметризованного case должен позволять понять вход без чтения кода. При необходимости задайте ids.

10 Тесты оценки

Функция метрик должна получать y_true, probabilities и threshold и проверять предусловия:

- длины совпадают;
- probabilities находятся в диапазоне от 0 до 1;
- threshold находится строго между 0 и 1;
- в y_true присутствуют оба класса.

Положительный тест проверяет ключи и диапазоны. Отрицательные используют pytest.raises(ValueError, match=...). Не закрепляйте слишком точное значение floating point без необходимости; используйте pytest.approx.

11 Тест model bundle

После обучения bundle обязан содержать:

- model с методом predict_proba;
- точный порядок features;
- версию формата;
- параметры обучения;
- границы split;
- validation-метрики.

Предсказание на корректной строке должно иметь форму (1, 2), а вероятность положительного класса — находиться в [0, 1]. Версия, которую возвращает API, должна совпадать с bundle.

12 Детерминированность

Обучите модель дважды с одинаковыми данными, параметрами и random_state. Сравните validation ROC AUC и probabilities на одном validation-фрагменте через pytest.approx.

Затем измените seed. Тест не обязан требовать другую метрику: на маленьком наборе результат может совпасть. Он должен проверять, что seed сохранён в bundle и отражён в run metadata.

13 Интеграционный smoke test

Минимальный путь:

1. проверить контракт;
2. обучить модель во временный файл;
3. загрузить bundle;
4. построить DataFrame в порядке features;

5. вызвать predict_proba;
6. проверить форму и диапазон;
7. не оставлять artifact вне tmp_path.

Интеграционный тест не заменяет unit-тесты: если он падает, unit-набор должен помочь локализовать слой.

14 Defect injection

Каждый участник временно вносит один контролируемый дефект в отдельной ветке или незакоммиченной копии:

- меняет порог цели >= 15 на > 15;
- меняет порядок FEATURES;
- удаляет проверку одного класса;
- разрешает DepHour=24;
- удаляет поле версии bundle.

Студент заранее называет тест, который должен упасть, запускает его по node ID и сохраняет фрагмент отчёта. После демонстрации дефект удаляется; в принимаемый tag он не входит.

15 Отчёты

Положительный отчёт:

```
python -m pytest -q --junitxml=reports/tests/lr5-pass.xml
```

Отрицательный отчёт сохраняется как небольшой текстовый фрагмент с командой, node ID, assertion и причиной. Не добавляйте огромный лог окружения и секреты.

Таблица рисков:

Риск	Тест	Ожидаемый сигнал
строка из будущего попала в train	chronological split test	assertion о границе времени
validation содержит один класс	evaluation precondition	ValueError
схема API и bundle расходятся	API contract test	неверная версия или 5xx
модельный файл повреждён	load smoke test	контролируемая ошибка загрузки

16 Индивидуальная сдача участника команды

Командный результат: tag lr5-v1, успешный JUnit XML и таблица рисков.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Роль	роль в ЛР5 и отличие от ЛР4
Задача	issue с критерием приёмки
Личный артефакт	test node ID, fixture, PR, review или отчёт
Контролируемый риск	ошибка, которую обнаруживает тест
Defect injection	внесённый дефект, ожидаемый и фактический failing test
Диагностика	первый полезный фрагмент traceback и причина
Воспроизведение	точная команда запуска одного теста и всего набора
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: tag или commit команды зафиксирован, личный test node ID существует, PR прошёл review другого участника, полный набор зелёный, отрицательный сценарий воспроизводится.

17 Защита

За 3–4 минуты участник:

1. запускает свой тест по node ID;
2. показывает риск и fixture;
3. временно активирует согласованный дефект;
4. объясняет assertion или исключение;
5. восстанавливает корректный код;
6. повторно получает успешный тест;
7. отвечает, почему соседний слой не заменяет эту проверку.

Преподаватель может назначить другой параметризованный case или попросить уменьшить fixture до минимального контрпримера.

18 Повторная сдача и стоп-факторы

Повторная сдача выполняется через новую issue или переоткрытую исходную, отдельный PR и обновлённую evidence card. История failed run сохраняется.

Стоп-факторы:

- тест всегда успешен или не содержит assertion;
- студент не может объяснить причину отказа;
- эталонные данные изменяются тестом;
- тест зависит от порядка или внешней сети без маркировки;
- дефект остаётся в принимаемой версии;
- отчёт сфальсифицирован или скопирован у другой команды.

19 Критерии оценивания

Критерий	Баллы	Полное выполнение
Fixtures и структура	2	изолированные fixtures и понятные слои тестов
Данные и контракт	2	параметризованные положительные и отрицательные cases
Модель и metrics	2	предусловия, bundle и диапазоны проверены
Интеграция и отчёты	1	smoke test и JUnit XML воспроизводимы
Командный процесс	1	issue, PR и содержательное review
Индивидуальная защита	2	defect injection, диагностика и объяснение риска

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

20 Чек-лист

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ Тесты не зависят от порядка выполнения.
- ☐ Временные файлы создаются через tmp_path.
- ☐ Есть минимум три параметризованных нарушения данных.
- ☐ Метрики отказывают при одном классе и неверных probabilities.
- ☐ Bundle содержит обязательные поля и загружается.

- ☐ Интеграционный smoke test проходит.
- ☐ JUnit XML создан автоматически.
- ☐ Каждый участник выполнил defect injection.
- ☐ PR прошёл review и создан tag lr5-v1.

21 Связь с ЛР6

В ЛР6 эти проверки войдут в контейнерную сборку. Docker image не принимается, если unit, contract и integration tests не проходят до build.

22 Официальные источники

- pytest documentation: https://docs.pytest.org/en/stable/
- pytest parametrization: https://docs.pytest.org/en/stable/how-to/parametrize.html
- scikit-learn metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html

Часть II. Сервис, CI и оркестрация

ЛР6. Контейнеризация и управление средой

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Предшествующая работа: ЛР5 «Тестирование данных, модели и pipeline»

1 Цель

Упаковать проверенную ML-систему в воспроизводимый OCI-образ, запустить её через Docker Compose с ограниченными правами и подтвердить готовность и предсказание реальными HTTP-запросами. Научиться различать build-time и runtime, image и container, health и readiness, воспроизводимый tag и плавающий latest.

2 Результаты обучения

После выполнения каждый студент умеет:

- объяснить build context, layer cache и назначение .dockerignore;
- читать стадии Dockerfile и локализовать ошибку сборки;
- выполнить тесты до создания runtime image;
- запускать процесс от non-root пользователя;
- передавать runtime-конфигурацию без включения секретов в образ;
- объяснить HEALTHCHECK, /healthz и /readyz;
- получить image ID и связать его с commit;
- выполнить smoke test контейнера и диагностировать один отказ;
- доказать личный вклад через issue, PR, review и evidence card.

3 Командный результат

Команда предоставляет tag lr6-v1 и:

1. многостадийный Dockerfile с quality/build gate;
2. .dockerignore с минимальным build context;
3. compose.yaml без поля version;
4. runtime от non-root пользователя;
5. healthcheck по /readyz;
6. read-only filesystem, tmpfs, cap_drop и no-new-privileges в Compose;

7. образ mlops-flight-delay:lr6-<short-sha>;
8. reports/container/lr6-inspect.txt с image ID, размером, пользователем и healthcheck;
9. reports/container/lr6-smoke.json с ответом /predict;
10. таблицу «риск — ограничение — проверка»;
11. issue, pull requests, review и индивидуальные evidence cards.

latest не принимается как идентификатор результата. Секреты, токены и персональные данные не помещаются в Dockerfile, build arguments, labels, image layers или отчёты.

4 Архитектура сборки

Стадия	Содержимое	Назначение
quality	исходный код, данные, tests, dev-зависимости	Ruff, pytest, контракт данных, обучение модели
runtime	runtime-зависимости, приложение, модель	минимальная среда исполнения API

Runtime image должен зависеть от успешной quality-стадии. Если тесты падают, принимаемый образ не создаётся. Учебная модель строится внутри build для воспроизводимости; в производственной системе модель обычно поступает как проверенный внешний артефакт через release pipeline.

5 Роли

Роль	Зона ответственности	Индивидуальное доказательство
Image engineer	стадии, cache и зависимости	diff Dockerfile и объяснение layers
Runtime security engineer	USER, filesystem и capabilities	Compose-ограничение и проверка inspect
Model artifact engineer	обучение и перенос модели	проверка bundle и readiness
Smoke-test engineer	health, predict и отчёт	скрипт или диагностический лог
Quality reviewer	.dockerignore, review и риски	содержательное review и defect injection

Для 2–4 человек роли совмещаются. Роль должна отличаться от ЛР5; один студент не остаётся единственным владельцем модели или инфраструктуры более двух последовательных работ.

6 Предварительные условия

- Docker Engine или Docker Desktop с Compose v2;
- успешный python -m pytest -q из ЛР5;
- закрытые данные и секреты удалены из build context;
- свободен порт 8080 либо задан APP_PORT;
- команда работает в отдельной ветке от tag ЛР5.

```
git switch -c lab6/containerization
docker version
docker compose version
python -m pytest -q
```

Если Docker недоступен на личной машине, преподаватель может предоставить общий учебный runner. Скриншот чужой сборки не заменяет индивидуальную диагностическую операцию.

7 Build context и .dockerignore

Build context — набор файлов, доступных инструкции COPY. Чем он больше, тем медленнее передача и выше риск включить лишние данные. Минимальный .dockerignore исключает:

```
.git
.github
.venv
```

```
__pycache__
.pytest_cache
.ruff_cache
docs
k8s
models
reports
mlruns
mlartifacts
*.pyc
.env
```

Для учебной quality-стадии src, tests, data, pyproject.toml и README.md остаются в context. Проверьте список до build; .dockerignore не является механизмом удаления уже попавшего секрета из истории образа.

8 Многостадийный Dockerfile

Обязательные решения:

- явная версия Python 3.12 slim;
- отдельная quality-стадия;
- установка зависимостей до часто меняющегося кода, где это возможно;
- Ruff, pytest и validation перед runtime;
- обучение models/model.joblib с фиксированными параметрами;
- runtime без tests, data и инструментов разработки;
- non-root USER с явным UID/GID;
- exec-форма CMD;
- HEALTHCHECK без внешней утилиты curl.

Сборка:

```
SHORT_SHA=$(git rev-parse --short HEAD)
docker build --pull --tag mlops-flight-delay:lr6-${SHORT_SHA} .
```

В PowerShell:

```
$shortSha = git rev-parse --short HEAD
docker build --pull --tag "mlops-flight-delay:lr6-$shortSha" .
```

Повторите build без --no-cache и сравните строки CACHED. Затем измените только README и объясните, какие layers перестроились.

9 Почему non-root обязателен

Если процесс не требует административных прав, он не должен работать как root. Проверьте конфигурацию image:

```
docker image inspect mlops-flight-delay:lr6-<short-sha> --format '{{.Config.User}}'
```

Ожидается непустой non-root UID/GID. Это снижает последствия компрометации, но не заменяет обновление base image, ограничение capabilities и безопасную конфигурацию хоста.

10 Health и readiness

/healthz подтверждает, что процесс отвечает. /readyz подтверждает, что модель загружена и сервис способен принимать предсказания. Docker healthcheck ЛР6 использует /readyz, иначе контейнер без модели мог бы считаться готовым.

```
docker inspect --format '{{json .State.Health}}' <container-id>
```

Статус сначала может быть starting, затем должен стать healthy. Не заменяйте ожидание фиксированным длинным sleep; опрашивайте состояние до timeout.

11 Запуск через Docker Compose

```
docker compose up --build --detach
docker compose ps
docker compose logs --no-color app
```

В compose.yaml должны быть:

- build и локальный неизменяемый для работы tag;
- настраиваемый host port ${APP_PORT:-8080};
- healthcheck;
- read_only: true;
- tmpfs: /tmp;
- cap_drop: ALL;
- security_opt: no-new-privileges:true;
- runtime environment без секретных значений.

Не добавляйте верхнеуровневое поле version: современная Compose Specification в нём не нуждается.

12 Smoke test

После статуса healthy выполните:

```
python scripts/container_smoke.py --base-url http://127.0.0.1:8080 --output reports/container/lr6-smoke.json
```

Скрипт обязан:

1. дождаться /readyz с ограниченным timeout;
2. отправить типизированный запрос /predict;
3. проверить HTTP status;
4. проверить диапазон delay_probability;
5. проверить непустые model_version и request_id;
6. сохранить компактный JSON без секретов.

Ручной curl полезен для диагностики, но принимаемый результат должен воспроизводиться одной командой.

13 Отчёт inspect

```
docker image inspect mlops-flight-delay:lr6-<short-sha> \
  --format 'id={{.Id}} size={{.Size}} user={{.Config.User}} health={{json .Config.Healthcheck}}'
```

Сохраните команду и результат в reports/container/lr6-inspect.txt. Image ID идентифицирует локальный content-addressed объект; для итогового release будет использован registry digest.

14 Проверка отсутствия лишних файлов

```
docker run --rm --entrypoint python mlops-flight-delay:lr6-<short-sha> -c \
  "from pathlib import Path; print([p.name for p in Path('/app').iterdir()])"
```

В runtime не должно быть .git, tests, исходных данных, .env, MLflow-хранилища и отчётов. Наличие файла внутри удалённого поздней инструкцией layer не означает, что он исчез из истории; не копируйте лишнее изначально.

15 Негативные сценарии

Команда воспроизводит минимум два отказа:

Дефект	Ожидаемый сигнал	Диагностика
модель не перенесена в runtime	/readyz возвращает 503, health unhealthy	проверить MODEL_PATH и COPY stage
процесс запущен на другом порту	health timeout	сопоставить CMD, EXPOSE и health URL
запрещена запись вне /tmp	операция завершается permission denied	проверить ожидаемый путь записи
порт хоста занят	Compose не запускает binding	изменить APP_PORT, не порт контейнера
.env попал в context	нарушение security review	исправить .dockerignore, сменить секрет

Defect injection выполняется в отдельной ветке или незакоммиченной копии и не входит в tag lr6-v1.

16 Таблица рисков

Риск	Ограничение	Проверка
непроверенный код попал в image	quality stage	намеренно падающий тест блокирует build
процесс получил лишние права	non-root, cap_drop, no-new-privileges	inspect и запуск команды id
образ содержит лишние данные	.dockerignore, выборочный COPY	просмотр /app и image history
сервис жив, но модель не готова	healthcheck по /readyz	Compose health status
результат нельзя связать с кодом	tag с short SHA и image ID	manifest inspect
runtime меняет файловую систему	read-only и tmpfs	контролируемая запись

17 Индивидуальная сдача участника команды

Командный результат: tag lr6-v1, image mlops-flight-delay:lr6-<short-sha>, успешные inspect и smoke reports.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР6, commit/tag и image ID
Роль	роль в ЛР6 и отличие от ЛР5
Задача	issue с наблюдаемым критерием приёмки
Личный артефакт	Dockerfile/Compose diff, smoke test, review или отчёт
Вклад	созданное, исправленное или проверенное решение
Инженерное решение	выбранный вариант и отвергнутая альтернатива
Риск	отказ, который выявляет healthcheck, inspect или smoke test
Диагностика	команда, первый полезный фрагмент лога и причина

Поле	Что указать
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: командная версия зафиксирована, образ связан с commit, личная issue закрыта через PR, review выполнен другим участником, контейнер healthy и индивидуальная проверка воспроизводится.

18 Защита

За 3–4 минуты участник:

1. показывает image tag, ID и связанный commit;
2. объясняет одну стадию или runtime-ограничение;
3. запускает собственную проверку;
4. воспроизводит назначенный отказ;
5. находит причину в build log, container log или inspect;
6. исправляет конфигурацию либо формулирует точное исправление;
7. объясняет, почему image нельзя идентифицировать только словом latest.

Преподаватель может изменить APP_PORT, удалить модель из build stage, назначить проверку пользователя или попросить показать файл, исключённый из context.

19 Повторная сдача и стоп-факторы

Повторная сдача выполняется через новую или переоткрытую issue, отдельный PR, новый image tag и обновлённую evidence card. Неуспешный отчёт сохраняется.

Стоп-факторы:

- контейнер работает от root без обоснованного исключения;
- используются latest или несвязанный с commit tag;
- тесты не являются gate сборки;
- /readyz успешен без загруженной модели;
- в image или отчёте обнаружен секрет;
- студент демонстрирует чужой образ или не может объяснить layer;
- дефект остался в принимаемом tag;
- smoke result отредактирован вручную.

20 Критерии оценивания

Критерий	Баллы	Полное выполнение
Build и воспроизводимость	2	стадии, test gate, фиксированный Python и связанный tag
Минимальный runtime	2	выборочный COPY, .dockerignore, нет tests/data/dev tools
Безопасность	2	non-root, read-only, tmpfs, capabilities и no-new-privileges
Health и smoke	1	ready healthcheck и автоматический predict report
Командный процесс	1	issue, небольшие PR и содержательное review
Индивидуальная защита	2	личный артефакт, отказ, диагностика и объяснение риска

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

21 Чек-лист

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ Build context не содержит секретов и лишних каталогов.

- ☐ Quality stage запускает Ruff, pytest и validation.
- ☐ Runtime image содержит обученную модель и приложение.
- ☐ Процесс работает от non-root UID/GID.
- ☐ Docker healthcheck проверяет /readyz.
- ☐ Compose использует read-only, tmpfs и security options.
- ☐ Образ помечен lr6-<short-sha>, а не latest.
- ☐ Inspect и smoke reports созданы командами.
- ☐ Каждый участник выполнил личную проверку и defect injection.
- ☐ PR прошёл review, создан tag lr6-v1.

22 Связь с ЛР7

В ЛР7 API-контракт будет расширен и проверен через OpenAPI, типизированные положительные и отрицательные запросы. Контейнерная граница ЛР6 остаётся неизменной: /healthz, /readyz, /predict, порт 8080 и модельный bundle являются совместным контрактом.

23 Официальные источники

- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Docker build best practices: https://docs.docker.com/build/building/best-practices/
- Docker build context and .dockerignore: https://docs.docker.com/build/concepts/context/
- Docker Compose services: https://docs.docker.com/reference/compose-file/services/
- Docker Compose startup and health: https://docs.docker.com/compose/how-tos/startup-order/

ЛР7. Сервис предсказаний и контракт API

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Предшествующая работа: ЛР6 «Контейнеризация и управление средой»

1 Цель

Сформировать явный и тестируемый HTTP-контракт сервиса предсказаний: типизированные запросы и ответы, документированные ошибки, OpenAPI-схему, request ID, readiness и набор положительных и отрицательных contract tests.

2 Результаты обучения

Каждый студент должен уметь:

- отличать API-контракт от внутренней реализации модели;
- описывать request и response через Pydantic;
- запрещать неизвестные поля и нежелательное приведение типов;
- читать OpenAPI и связывать JSON Schema с тестом;
- различать 404, 422, 500 и 503;
- обеспечивать request ID в успешных и ошибочных ответах;
- тестировать API через TestClient и реальный HTTP-контур;
- объяснять совместимость изменения контракта;
- диагностировать назначенный негативный сценарий;
- подтверждать личный вклад независимо от результата команды.

3 Командный результат

Команда предоставляет tag lr7-v1 и:

1. FastAPI-сервис версии 0.3.0;
2. строгую PredictionRequest без неизвестных полей;
3. типизированную PredictionResponse;
4. единый ErrorResponse для 422 и 503;
5. X-Request-ID в header и request_id в body;
6. /healthz, /readyz, /predict, /metrics, /openapi.json и /docs;
7. сохранённую схему reports/openapi/lr7-openapi.json;
8. pytest contract tests OpenAPI, входов, выходов и ошибок;
9. HTTP-отчёт reports/api/lr7-contract-smoke.json минимум с четырьмя cases;
10. таблицу совместимости изменений;
11. issue, небольшие PR, review и индивидуальные evidence cards.

Сервис принимает только признаки, доступные за 60 минут до планового отправления. Фактическая задержка и другие поля будущего не могут быть частью request.

4 Контракт endpoints

Метод и путь	Назначение	Успех	Основные ошибки
GET /healthz	процесс отвечает	200	5xx инфраструктуры
GET /readyz	модель загружена	200	503
POST /predict	получить прогноз	200	422, 503
GET /metrics	технические метрики	200	5xx
GET /openapi.json	машиночитаемый контракт	200	5xx
GET /docs	интерактивная документация	200	5xx

/healthz не выполняет модельный прогноз. /readyz не подменяет contract test /predict.

5 Request schema

```
{
  "dep_hour": 9,
  "is_weekend": 0
}
```

Требования:

- dep_hour — integer от 0 до 23;
- is_weekend — integer 0 или 1;
- оба поля обязательны;
- строки "9", boolean и float не преобразуются неявно в integer;
- неизвестные поля запрещены;
- actual_delay, ArrDelay и иные признаки будущего отвергаются.

Строгость является решением контракта. Если клиенту нужна допустимая конверсия, она вводится явно, документируется и тестируется.

6 Response schema

Успешный ответ:

```
{
  "delay_probability": 0.37,
  "delayed": false,
  "threshold": 0.5,
  "model_version": "0.2.0",
  "request_id": "lr7-demo"
}
```


Инварианты:

- delay_probability находится в [0, 1];
- delayed == (delay_probability >= threshold);
- threshold явно присутствует;
- model_version соответствует bundle;
- request_id непустой и совпадает с X-Request-ID.

Response model ограничивает и документирует выход. Не возвращайте внутренний объект модели, stack trace, путь файловой системы или входные секреты.

7 Error schema


```
{
  "error_code": "validation_error",
  "message": "Запрос не соответствует контракту",
  "request_id": "lr7-demo",
  "details": [
    {
      "location": ["body", "dep_hour"],
      "message": "Input should be less than or equal to 23",
      "error_type": "less_than_equal"
    }
  ]
}
```


Ошибка должна помогать клиенту исправить запрос, но не раскрывать stack trace или данные других запросов. Текст Pydantic может меняться между версиями; тестируйте стабильные поля error_code, status, location и наличие диагностической детали, а не полный текст сообщения.

8 Семантика status codes

Status	Значение в ЛР7
200	запрос выполнен по контракту
404	endpoint не существует
422	JSON получен, но не соответствует schema
500	необработанная внутренняя ошибка; считается дефектом
503	процесс жив, но модель не готова

Не возвращайте 200 с полем "error". HTTP status является частью машинного контракта.

9 Request ID

Middleware принимает непустой X-Request-ID разумной длины или создаёт UUID. Значение возвращается в response header. Для JSON-ответов /predict и ошибок оно также включается в body.

Request ID нужен для связи клиентского симптома, лога и метрики. Он не должен содержать персональные данные, token или полный payload.

Проверка:

```
curl -i -X POST http://127.0.0.1:8080/predict \
  -H 'Content-Type: application/json' \
  -H 'X-Request-ID: student-case-1' \
  -d '{"dep_hour":9,"is_weekend":0}'
```

10 Lifespan модели

Модель загружается один раз в lifespan перед обслуживанием запросов, а не на каждый /predict. Это сокращает latency и задаёт ясный readiness contract.

Если файл модели отсутствует, процесс может отвечать на /healthz, но /readyz и /predict возвращают 503. Если файл существует, но повреждён, startup должен завершиться наблюдаемой ошибкой, а не принимать запросы с частично инициализированным состоянием.

11 OpenAPI как исполняемая спецификация

Сохраните схему:

python scripts/export_openapi.py --output reports/openapi/lr7-openapi.json

Проверьте:

- info.version равен 0.3.0;
- /predict содержит POST;
- request body ссылается на PredictionRequest;
- additionalProperties равно false;
- оба поля перечислены в required;
- 200, 422 и 503 имеют JSON schema;
- response ссылается на PredictionResponse;
- error responses ссылаются на ErrorResponse.

Схема, сгенерированная из незакоммиченного кода, не является доказательством принимаемой версии.

12 Unit contract tests

Обязательные pytest cases:

1. корректный запрос возвращает 200 и полный response;
2. dep_hour=24 возвращает 422;
3. отсутствующее поле возвращает 422;
4. неизвестное поле возвращает 422;
5. строка вместо integer возвращает 422;
6. при отсутствующей модели /readyz и /predict возвращают 503;
7. request ID совпадает в header и body;
8. OpenAPI содержит request, response и error schemas.

Для отдельных значений используйте параметризацию. Тест не должен зависеть от запущенного Docker container; он использует TestClient и временную модель.

13 HTTP contract smoke

После запуска Compose:

```
python scripts/api_contract_smoke.py \
  --base-url http://127.0.0.1:8080 \
  --output reports/api/lr7-contract-smoke.json
```

Скрипт выполняет cases:

- valid — 200;
- hour_out_of_range — 422;
- unknown_field — 422;
- wrong_type — 422.

Каждый результат содержит request, ожидаемый и фактический status, response и X-Request-ID. Отчёт создаётся автоматически и не редактируется вручную.

14 Проверка OpenAPI в Swagger UI

Откройте /docs и проверьте пример request, обязательность полей, диапазоны и перечисленные responses. Затем сравните с lr7-openapi.json.

Swagger UI помогает исследовать контракт, но скриншот не заменяет сохранённую schema и автоматические tests. В production документация может быть закрыта политикой доступа, однако сам контракт остаётся доступен pipeline.

15 Совместимость изменений

Изменение	Обычно совместимо?	Действие
добавить необязательное поле ответа	да, если клиент игнорирует неизвестное	обновить schema и tests
удалить поле ответа	нет	новая версия API или миграция
сузить допустимый диапазон	нет для прежних клиентов	согласовать переход
добавить обязательное поле request	нет	версионирование и период миграции
добавить новый endpoint	да	документировать
изменить смысл существующего поля	нет	новое имя или версия

Совместимость определяется поведением клиентов, а не только тем, что сервер запускается.

16 Негативные сценарии

Каждый участник воспроизводит один контролируемый отказ:

- разрешить неизвестные поля;
- убрать strict mode и принять строку "9";
- удалить response model;
- перестать возвращать request ID в header;
- сделать /readyz успешным без модели;
- убрать документированный response 503 из OpenAPI.

До изменения студент называет тест, который должен упасть. Дефект не входит в tag lr7-v1.

17 Таблица рисков

Риск	Контракт	Проверка
клиент отправляет признак будущего	extra forbid	unknown_field case возвращает 422
тип преобразован молча	strict input	строка вместо integer возвращает 422
ответ изменился незаметно	response model и OpenAPI	schema test
модель не загружена	readiness	503 и error_code not_ready

Риск	Контракт	Проверка
запрос нельзя найти в логах	request ID	header равен body
клиент зависит от текста ошибки	stable error_code	tests не сравнивают весь message

18 Индивидуальная сдача участника команды

Командный результат: tag lr7-v1, OpenAPI JSON, успешный pytest и HTTP contract-smoke.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР7, commit/tag и image ID ЛР6/ЛР7
Роль	роль в ЛР7 и отличие от ЛР6
Задача	issue с наблюдаемым критерием приёмки
Личный артефакт	model schema, endpoint, test, smoke case, review или OpenAPI-проверка
Вклад	созданное, исправленное или проверенное изменение
Инженерное решение	выбранная строгость или форма контракта и альтернатива
Риск	несовместимость или отказ, обнаруживаемый тестом
Диагностика	status, request ID, test node ID и причина
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: командная версия зафиксирована, OpenAPI соответствует commit, личная issue закрыта через PR, review выполнен другим участником, tests зелёные, личный negative case воспроизводится.

19 Защита

За 3–4 минуты участник:

1. показывает свой элемент OpenAPI или код контракта;
2. выполняет положительный запрос;
3. выполняет назначенный отрицательный запрос;
4. связывает status, error_code и request ID;
5. находит проверяющий test node ID;
6. объясняет совместимость одного изменения;
7. восстанавливает корректную версию после defect injection.

Преподаватель может добавить неизвестное поле, заменить integer строкой, убрать модель или попросить найти schema конкретного response.

20 Повторная сдача и стоп-факторы

Повторная сдача выполняется через issue, отдельный PR, обновлённую OpenAPI schema, новый smoke report и evidence card. История первой попытки сохраняется.

Стоп-факторы:

- endpoint принимает произвольный dict вместо типизированной модели;
- поле будущего принимается без ошибки;
- error возвращается со status 200;
- /readyz успешен без модели;
- request ID отсутствует или содержит секрет;

- schema не соответствует принимаемому commit;
- negative tests отсутствуют или всегда зелёные;
- студент не может объяснить собственное поле или status;
- отчёт изменён вручную.

21 Критерии оценивания

Критерий	Баллы	Полное выполнение
Request contract	2	типы, диапазоны, strict и extra forbid
Response и errors	2	типизированный ответ, error model и status semantics
OpenAPI	2	schema сохранена и автоматически проверена
Request ID и readiness	1	header/body согласованы, 503 воспроизводим
Командный процесс	1	issue, небольшие PR и содержательное review
Индивидуальная защита	2	личный артефакт, negative case и совместимость

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

22 Чек-лист

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ Request и response описаны Pydantic-моделями.
- ☐ Неизвестные поля и неявные типы отвергаются.
- ☐ 422 и 503 имеют единый ErrorResponse.
- ☐ Request ID совпадает в header и body.
- ☐ /readyz зависит от загруженной модели.
- ☐ OpenAPI сохранён из принимаемого commit.
- ☐ Pytest содержит положительные и отрицательные cases.
- ☐ HTTP smoke report содержит минимум четыре cases.
- ☐ Каждый участник выполнил defect injection и диагностику.
- ☐ PR прошёл review, создан tag lr7-v1.

23 Связь с ЛР8

В ЛР8 Ruff, pytest, data validation, OpenAPI export и container build станут обязательными quality gates GitHub Actions. Pull request не может быть принят, если contract tests или проверка schema завершились неуспешно.

24 Официальные источники

- FastAPI request body and models: https://fastapi.tiangolo.com/tutorial/body/
- FastAPI response models: https://fastapi.tiangolo.com/tutorial/response-model/
- FastAPI error handling: https://fastapi.tiangolo.com/tutorial/handling-errors/
- FastAPI lifespan: https://fastapi.tiangolo.com/advanced/events/
- FastAPI OpenAPI: https://fastapi.tiangolo.com/tutorial/metadata/
- Pydantic models and configuration: https://docs.pydantic.dev/latest/concepts/models/

ЛР8. Непрерывная интеграция и quality gates

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Предшествующая работа: ЛР7 «Сервис предсказаний и контракт API»

1 Цель

Преобразовать локальные проверки проекта в воспроизводимый GitHub Actions pipeline, который автоматически блокирует принятие дефектного изменения, сохраняет диагностические артефакты и даёт проверяемые доказательства личной работы каждого участника.

2 Результаты обучения

Каждый студент должен уметь:

- объяснять события, jobs, steps, actions и runners;
- различать локальную проверку и обязательный remote check;
- настраивать минимальные permissions и отмену устаревших запусков;
- проектировать порядок быстрых и дорогих gates;
- читать log, exit code и JUnit/JSON-отчёт;
- сохранять диагностические artifacts без секретов;
- воспроизводить контролируемый отказ и исправлять корневую причину;
- связывать run, commit, pull request и правило ветки;
- подтверждать личный вклад независимо от результата команды.

3 Командный результат

Команда предоставляет tag lr8-v1 и:

1. workflow .github/workflows/ci.yml;
2. события pull_request, push в main и ручной workflow_dispatch;
3. read-only permission для contents;
4. concurrency с отменой устаревшего run той же ветки;
5. job quality с lint, format check, data validation, tests, OpenAPI export и smoke training;
6. job container, зависящий от успешного quality;
7. image tag по полному ${{ github.sha }}, без latest;
8. diagnostic artifact с JUnit, data-validation и OpenAPI reports;
9. required checks quality и container для main;
10. один намеренно неуспешный и один успешный run на каждого участника;
11. issue, небольшие PR, review и индивидуальные evidence cards.

4 Модель GitHub Actions

Понятие	Назначение	Пример в ЛР8
event	причина запуска	pull_request
workflow	файл автоматизации	ci.yml
job	изолированный набор шагов	quality
step	команда или action	python -m pytest
action	переиспользуемая операция	actions/checkout
runner	машина выполнения	ubuntu-latest
artifact	сохранённый результат run	JUnit и OpenAPI
check	статус job в pull request	quality

Каждый job получает чистый runner. Наличие файла на ноутбуке или в другом job нельзя предполагать без явной передачи artifact или повторного построения.

5 События запуска

Workflow запускается:

- при открытии и обновлении pull request;
- при push в main, чтобы проверить принятую историю;
- вручную через workflow_dispatch для демонстрации и диагностики.

Не используйте pull_request_target для исполнения кода из недоверенного fork: этот event имеет иной security context и при ошибочной конфигурации может открыть secrets или write token. Для учебного CI достаточно pull_request.

6 Permissions и supply-chain граница

В начале workflow задаётся:

```
permissions:
  contents: read
```

Token получает только необходимое право чтения. Checkout выполняется с persist-credentials: false, поскольку последующие steps не должны выполнять push.

Используйте проверенные официальные actions и зафиксированные major versions. На начало подготовки материалов используются actions/checkout@v7, actions/setup-python@v7 и actions/upload-artifact@v4. Перед запуском потока преподаватель повторно сверяет поддерживаемые версии и changelog; номер в методичке не является бессрочной гарантией.

В зрелом pipeline action дополнительно закрепляют полным commit SHA. В ЛР8 студент должен объяснить разницу между major tag и immutable SHA.

7 Concurrency

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

Если участник быстро отправляет несколько commits в одну ветку, предыдущий run уже не доказывает качество актуального commit. Отмена экономит runner time и уменьшает очередь. Run другой ветки не должен отменяться, поэтому group включает ref.

8 Порядок quality gates

Job quality выполняет:

1. checkout;
2. установку Python с pip cache;
3. установку проекта и dev-зависимостей;
4. ruff check;
5. ruff format --check;
6. проверку data contract;
7. pytest с JUnit report;
8. экспорт OpenAPI;
9. smoke training;
10. загрузку reports как artifact.

Быстрые статические проверки расположены раньше дорогих. Ошибка прекращает дальнейшие обычные steps, но upload выполняется с if: always(), чтобы сохранить уже созданную диагностику.

9 Container gate

Job container содержит needs: quality. Дорогая сборка начинается только после успешных проверок кода, данных и API.

```
- name: Build immutable image tag
  run: docker build --tag mlops-flight-delay:${{ github.sha }} .
```

SHA связывает image с конкретным commit. Tag latest изменяем и не является доказательством происхождения. В ЛР8 push в registry ещё не выполняется: публикация без политики доступа и release process преждевременна.

10 Diagnostic artifacts

Workflow сохраняет каталог reports/ на 14 дней. Минимум:

- reports/validation/ci.json;
- reports/tests/ci.xml;
- reports/openapi/ci-openapi.json.

Artifact предназначен для диагностики и доказательства, а не для постоянного хранения модели. Не загружайте .env, tokens, полный production dataset, персональные данные и необработанные request payloads.

Имя включает github.run_id, поэтому evidence card однозначно ссылается на run. Студент скачивает artifact и проверяет содержимое, а не ограничивается зелёной иконкой.

11 Проверка CI-контракта

Тест tests/test_ci_contract.py защищает ключевые решения workflow: permissions, concurrency, обязательные команды, artifact upload, зависимость jobs и immutable image tag. Он также запрещает pull_request_target и latest.

Текстовый contract test не доказывает полную корректность YAML и GitHub semantics. Он служит ранней защитой от учебного регресса и дополняется реальным Actions run.

12 Настройка защиты main

В branch protection или ruleset настройте:

- изменения только через pull request;
- минимум одно одобрение не от автора;
- разрешение обсуждений до merge;
- required status checks quality и container;
- запрет force push и удаления main;
- отсутствие обхода правил в учебном процессе.

Имена checks должны быть уникальны. Если одинаковое имя появляется в нескольких workflows, GitHub может показывать неоднозначный статус required check.

Если тариф или права не позволяют применить настройку, команда документирует ограничение и демонстрирует эквивалентный обязательный процесс. Это снижает полноту автоматизации и должно быть явно отражено в evidence.

13 Порядок выполнения

Этап 1. Подготовка

1. Синхронизируйте main и создайте issue.
2. Создайте отдельную ветку.
3. Выполните локально все команды из PR template.
4. Убедитесь, что generated reports не содержат секретов.

Этап 2. Положительный pipeline

1. Добавьте workflow.
2. Создайте pull request.
3. Дождитесь quality и container.
4. Откройте каждый step, зафиксируйте run ID и commit SHA.
5. Скачайте artifact и проверьте три обязательных файла.

Этап 3. Индивидуальный controlled failure

Каждый участник в собственной ветке вводит один небольшой обратимый дефект, заранее указывает ожидаемый упавший step и отправляет commit. После красного run участник фиксирует ссылку и диагностику, исправляет причину отдельным commit и получает зелёный run.

Этап 4. Branch protection

Настройте required checks и покажите, что merge недоступен при красном quality или container. После успешных checks и review выполните merge по принятой стратегии.

Этап 5. Фиксация версии

На принятом commit создайте tag lr8-v1. Ссылки evidence должны вести на runs соответствующей ветки и итоговый commit.

14 Варианты контролируемого отказа

Дефект	Ожидаемый step	Диагностическое доказательство
нарушение Ruff	Lint	файл, строка и rule code
неотформатированный файл	Check formatting	diff formatter
неверный тип в данных	Validate data contract	JSON report и violation
сломанный инвариант теста	Run tests	test node ID и JUnit failure
удалённый response schema	Run tests или Export OpenAPI	упавший contract test
синтаксическая ошибка Dockerfile	Build immutable image tag	build stage и error
отсутствующий файл в build context	container	COPY error

Участники одной команды выбирают разные дефекты. Нельзя подменять эксперимент ручной отменой run или временным отключением required check.

15 Диагностика красного run

Используйте последовательность:

1. проверить event, branch, commit SHA и отмену устаревшего run;
2. найти первый упавший step, а не последний пропущенный;
3. прочитать команду и exit code;
4. открыть аннотацию или report;
5. воспроизвести команду локально;
6. сформулировать корневую причину одним проверяемым предложением;
7. внести минимальное исправление;
8. сопоставить новый run с исправляющим commit.

Кнопка Re-run полезна для внешней нестабильности, но не является исправлением детерминированного дефекта.

16 Матрица командных ролей

Для команды 2–5 человек роли объединяются или распределяются:

- владелец quality job;
- владелец container job;
- владелец diagnostic artifacts;
- владелец branch rules и PR process;
- reviewer CI/security contract.

В следующей ЛР роли ротируются. Один участник не должен единолично написать workflow и раздать остальным одинаковые screenshots.

17 Индивидуальная сдача участника команды

Командный результат: tag lr8-v1, защищённый main, успешные checks и diagnostic artifact.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР8, итоговый commit/tag
Роль	роль в ЛР8 и отличие от ЛР7
Задача	issue с наблюдаемым критерием приёмки
Личный артефакт	workflow step, test, report, правило или review
Негативный run	ссылка, run ID, commit и ожидаемый failed step
Диагностика	первая полезная ошибка, exit code/report и причина
Исправление	отдельный commit и успешный run
Artifact	имя, срок хранения и проверенные файлы
Инженерное решение	порядок gates или permission и альтернатива
Риск	CI/supply-chain риск и мера
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: личная issue закрыта через PR, negative run сохранён, причина объяснена, исправление подтверждено green run, review выполнен другим участником, итоговые required checks относятся к принимаемому commit.

18 Защита

За 3–4 минуты участник:

1. показывает свою issue и изменение;
2. открывает намеренно неуспешный run;
3. называет первый упавший step и корневую причину;
4. открывает исправляющий commit и зелёный run;
5. скачивает или открывает свой report из artifact;
6. объясняет, почему container не стартовал до quality;
7. показывает required check и невозможность merge при отказе;
8. объясняет один риск token/action/artifact.

Преподаватель может попросить определить, какой run относится к HEAD pull request, изменить data contract, удалить response schema или объяснить, почему re-run без commit не исправляет код.

19 Повторная сдача и стоп-факторы

Повторная сдача выполняется через новую issue или переоткрытый PR, отдельный исправляющий commit, новые runs и обновлённую evidence card. История первой попытки сохраняется.

Стоп-факторы:

- workflow запускается только вручную;
- merge выполнен с красным required check;
- contents: write задан без обоснования;
- используется pull_request_target для выполнения student code;
- image помечен только latest;
- reports или logs содержат secret/персональные данные;
- negative run общий на команду либо создан отменой;
- студент не может связать run с commit;
- успешный статус получен отключением проверки;
- evidence отредактировано вручную без происхождения.

20 Критерии оценивания

Критерий	Баллы	Полное выполнение
Workflow и события	2	PR, main, manual; permissions и concurrency корректны
Quality gates	2	lint, format, data, tests, OpenAPI, training выполняются
Container и происхождение	1	зависимость от quality и tag по SHA
Artifacts и диагностика	1	отчёты сохраняются и читаются участником
Защита ветки и review	1	required checks и содержательное review
Индивидуальный failure/fix	3	отдельные runs, причина, исправление и устная защита

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

21 Чек-лист

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ Workflow запускается на pull request, push в main и вручную.
- ☐ Token имеет только contents: read.
- ☐ Устаревший run ветки отменяется concurrency policy.
- ☐ quality содержит все обязательные проверки.
- ☐ JUnit, validation и OpenAPI входят в artifact.
- ☐ container зависит от quality.
- ☐ Image tag содержит полный commit SHA.
- ☐ quality и container являются required checks.
- ☐ Каждый участник создал собственный negative run.
- ☐ Каждый negative run исправлен отдельным commit.
- ☐ Evidence card содержит ссылки, run ID и диагностику.
- ☐ Итоговая версия отмечена tag lr8-v1.

22 Связь с ЛР9

В ЛР9 pipeline будет расширен публикацией версионированного image и безопасной работой с registry. ЛР8 создаёт доверенную границу: публикация возможна только после проверок кода, данных, API и container build.

23 Официальные источники

- GitHub Actions workflow syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
- GitHub Actions concurrency: https://docs.github.com/en/actions/concepts/workflows-and-actions/concurrency
- Workflow artifacts: https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts
- Protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
- actions/checkout releases: https://github.com/actions/checkout/releases
- actions/setup-python releases: https://github.com/actions/setup-python/releases
- actions/upload-artifact: https://github.com/actions/upload-artifact

ЛР9. Оркестрация обучения и обработка отказов

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Предшествующая работа: ЛР8 «Непрерывная интеграция и quality gates»

1 Цель

Преобразовать последовательность обучения модели в наблюдаемый и параметризованный Prefect 3 flow validate -> preprocess -> train -> evaluate, обеспечить идемпотентные артефакты, осмысленные retries, quality gate и диагностику контролируемых отказов.

2 Результаты обучения

Каждый студент должен уметь:

- различать код обработки и код оркестрации;
- декомпозировать pipeline на tasks с явными входами и выходами;
- задавать типизированные параметры flow;
- читать состояния flow run и task run;
- применять retry только к потенциально временной ошибке;
- объяснять timeout, cache и идемпотентность;
- сохранять отчёты до возбуждения ошибки;
- связывать run с данными, параметрами, моделью и метриками;
- воспроизводить и диагностировать индивидуальный отказ.

3 Командный результат

Команда предоставляет tag lr9-v1 и:

1. Prefect 3 flow flight-delay-training;
2. tasks validate, preprocess, train, evaluate;
3. CLI-параметры путей, гиперпараметров, threshold и minimum ROC AUC;
4. retry policy для стадии подготовки и отсутствие бессмысленных retries у deterministic gates;
5. каталоги минимум двух runs: успешного и контролируемого отказа;

6. JSON-отчёты каждой завершённой стадии;
7. run-summary.json успешного запуска;
8. тесты чистых stage-функций и отрицательного quality gate;
9. CI smoke запуска flow;
10. issue, небольшие PR, review и индивидуальные evidence cards.

4 Установка

```
python -m pip install -e ".[dev,orchestration]"
prefect version
```

На дату актуализации используется Prefect 3.8.x и диапазон зависимости prefect>=3.8,<4. Перед семестром преподаватель сверяет актуальную стабильную версию. Установка Prefect Cloud не обязательна: flow должен работать локально с ephemeral API или настроенным локальным server.

5 Граница обработки и оркестрации

Чистые функции validate_stage, preprocess_stage, train_stage, evaluate_stage выполняют предметную работу и тестируются без UI. Decorated tasks добавляют состояния, retries и logs. Flow определяет зависимости и общий run.

Не помещайте всю обработку внутрь одного @flow: тогда невозможно независимо повторить, диагностировать и оценить стадии. Не переносите расчёт метрик в YAML или deployment configuration.

6 Граф зависимостей

```
data + contract
      |
   validate
      |
  preprocess
      |
    train
      |
   evaluate
      |
run-summary.json
```

Последовательность обусловлена данными: train получает подготовленный CSV, evaluate — подготовленный CSV и model bundle. Порядок не должен зависеть от случайной очередности файловой системы.

7 Запуск успешного flow

```
python -m mlops_flight_delay.orchestration \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --run-dir reports/prefect/lr9-success \
  --model models/lr9-model.joblib \
  --n-estimators 80 \
  --max-depth 4 \
  --random-state 42 \
  --minimum-roc-auc 0.5
```

В PowerShell обратный слеш замените обратным апострофом либо выполните команду одной строкой.

Успешный каталог содержит:

- 01-validation.json;
- prepared.csv;
- 03-training.json;
- 04-evaluation.json;
- run-summary.json.

8 Явные параметры

Путь к данным, контракту, каталогу run и модели не зашивается в task. Гиперпараметры и quality threshold также передаются явно.

Параметризация обеспечивает:

- повторяемость запуска;
- сравнение runs;
- тестирование граничных значений;
- будущий deployment без изменения кода;
- связь решения с evidence card.

Не включайте secrets в параметры командной строки: они попадают в историю shell и metadata run.

9 Состояния

Prefect фиксирует состояния, включая Pending, Running, Completed, Failed и Retrying. Важна причина перехода, а не цвет UI.

Если evaluate возбуждает исключение, task и flow получают Failed. Если preprocess временно не может прочитать ресурс и исчерпал retries, итог также Failed. Skipped downstream task не является корневой причиной.

Для evidence участник записывает run name/ID, параметры, первую failed task, тип исключения и восстановление.

10 Retry policy

В starter flow preprocess имеет две повторные попытки с небольшой задержкой. Это учебная демонстрация поведения при временном сбое ввода.

Не повторяйте автоматически:

- нарушение data contract;
- недопустимый параметр;
- недостаточную метрику модели;
- отсутствующий обязательный столбец;
- программный дефект.

Retry не исправляет детерминированную ошибку и только увеличивает время и стоимость. В зрелой версии используйте условие retry по классу исключения и экспоненциальную задержку для внешнего сервиса.

11 Контролируемый retry

```
python -m mlops_flight_delay.orchestration \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --run-dir reports/prefect/lr9-retry-student-01 \
  --model models/lr9-retry-student-01.joblib \
  --simulate-transient-failure
```

На первой попытке preprocess создаёт marker .retry-once и возбуждает OSError; следующая попытка продолжает работу. Для повторной демонстрации используйте новый run directory. Marker является частью учебного сценария, а не общим паттерном production-кода.

12 Детерминированный quality-gate отказ

```
python -m mlops_flight_delay.orchestration \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --run-dir reports/prefect/lr9-quality-fail \
  --model models/lr9-quality-fail.joblib \
  --minimum-roc-auc 1.1
```

evaluate сначала записывает 04-evaluation.json с passed: false, затем возбуждает ошибку. Такой отказ не повторяется: порог 1.1 недостижим для ROC AUC и требует исправления параметра или явного решения владельца, а не retry.

13 Идемпотентность

Повтор одного запуска с теми же входами и random seed должен давать тот же подготовленный dataset hash и метрики в пределах детерминированного алгоритма. Task не должна бесконтрольно дописывать строки в общий файл.

В ЛР9 output directory задаётся явно. Для нового эксперимента создаётся новый каталог; для осознанного повторения допускается атомарная перезапись собственных файлов. Общий mutable latest/ запрещён как единственное evidence.

14 Артефакты и происхождение

run-summary.json связывает:

- название и порядок стадий;
- SHA256 исходных и подготовленных данных;
- путь model bundle;
- параметры обучения;
- test metrics;
- итог quality gate.

Сам путь не является глобальным идентификатором. В следующих ЛР manifest будет дополнен MLflow run/model version и registry information.

15 Logs

Task logs содержат количество строк, hash и метрики, но не полный dataset. Исключение должно объяснять действие: какой gate не пройден и где лежит report.

Не логируйте token, .env, персональные данные, полный request payload и содержимое закрытого dataset. Prefect UI облегчает поиск, но политика данных остаётся ответственностью команды.

16 Timeout и cache

Timeout ограничивает зависшую работу и должен учитывать нормальное время стадии. Слишком короткий timeout создаёт ложные отказы, слишком большой задерживает реакцию.

Cache может пропустить повторное вычисление при совпавшем ключе. Он вводится только после определения состава ключа: версия кода, hash данных, параметры и relevant environment. Cache по имени файла недостаточен. В обязательном starter flow cache намеренно не включён, чтобы студент сначала доказал идемпотентность.

17 Тестирование

Обязательные tests:

1. положительная последовательность создаёт четыре stage artifacts;
2. model bundle содержит заданный random seed;
3. evaluation report имеет четыре метрики;
4. недостижимый quality threshold вызывает ValueError;
5. отрицательный report записан до ошибки;
6. retry policy preprocess равна двум попыткам;
7. CI выполняет flow smoke.

Тестирование чистых stage-функций быстро локализует дефект. Отдельный integration smoke доказывает, что decorators и flow wiring исполняются.

18 Индивидуальные failure-сценарии

Каждый участник выбирает отдельный сценарий:

Сценарий	Ожидаемая task	Ожидаемое поведение
controlled transient input	preprocess	Retrying, затем Completed
отсутствующий столбец	validate или preprocess	Failed без полезного retry
повреждённая дата	validate/preprocess	Failed с диагностикой поля
threshold ROC AUC 1.1	evaluate	report passed=false, Failed
путь model недоступен	train	Failed, downstream не выполняется
неверный тип параметра	flow validation	запуск отклонён до обработки

Сценарий не должен повреждать общий main или удалять чужие artifacts.

19 Командные роли

Для команды 2–5 человек роли объединяются или распределяются:

- владелец validate/data contract;
- владелец preprocess и идемпотентности;
- владелец train/parameters;
- владелец evaluate/quality gate;
- владелец orchestration tests и failure review.

Роль координатора без технического или проверочного артефакта не считается самостоятельным вкладом. Роли меняются относительно ЛР8.

20 Индивидуальная сдача участника команды

Командный результат: tag lr9-v1, flow, tests, успешный run и набор failure runs.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР9, commit/tag
Роль	роль в ЛР9 и отличие от ЛР8
Задача	issue с наблюдаемым критерием приёмки
Личный артефакт	task, параметр, test, report или review
Успешный run	имя/ID, параметры и итоговый summary
Отказ	личный сценарий, состояние, первая ошибка
Retry	число попыток и обоснование применимости
Происхождение	data hash, model path, parameters и metrics

Поле	Что указать
Решение	выбранная граница task и альтернатива
Риск	повтор, частичный output, cache или secret
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: личная issue закрыта через PR, review выполнен другим участником, успешный run воспроизводим, failure run индивидуален, JSON evidence соответствует commit, участник объясняет retry и идемпотентность.

21 Защита

За 3–4 минуты участник:

1. показывает свою task или test в tag lr9-v1;
2. запускает flow с изменённым безопасным параметром;
3. открывает состояние и JSON-артефакт;
4. воспроизводит назначенный failure;
5. называет первую failed task и downstream effect;
6. объясняет, нужен ли retry;
7. связывает data hash, параметры, модель и metric;
8. показывает исправление или восстановление.

Преподаватель может изменить threshold, random seed, путь данных или попросить классифицировать новую ошибку как transient/deterministic.

22 Повторная сдача и стоп-факторы

Повторная сдача выполняется через issue, отдельный PR, новые run directory и evidence card. История первой попытки сохраняется.

Стоп-факторы:

- весь pipeline оформлен одной task;
- параметры зашиты в код и не видны в run;
- нарушение data contract повторяется retries и скрывается;
- failed report не сохранён;
- flow считается успешным после failed quality gate;
- общий mutable каталог перезаписывает evidence других участников;
- logs содержат secret или закрытые данные;
- один failure-сценарий выдан всей команде;
- студент не может связать run с commit и входами;
- скриншот UI заменяет код и reports.

23 Критерии оценивания

Критерий	Баллы	Полное выполнение
Декомпозиция flow	2	четыре tasks, явные зависимости и границы
Параметры и происхождение	2	входы, hash, параметры, model и metrics связаны
Отказы и retries	2	transient retry и deterministic failure различаются
Artifacts и идемпотентность	1	reports создаются безопасно и воспроизводимо
Тесты и CI	1	положительный, отрицательный и flow smoke

Критерий	Баллы	Полное выполнение
Индивидуальная защита	2	личная task/failure, диагностика и решение

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

24 Чек-лист

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ Используется Prefect 3 из проверенного диапазона.
- ☐ Flow содержит validate, preprocess, train и evaluate.
- ☐ Все пути и параметры передаются явно.
- ☐ Validation failure останавливает downstream.
- ☐ Preprocess retry демонстрируется контролируемо.
- ☐ Deterministic quality failure не маскируется retry.
- ☐ Каждая стадия создаёт проверяемый report.
- ☐ Успешный run содержит run-summary.json.
- ☐ Tests проверяют положительный и отрицательный сценарии.
- ☐ CI выполняет Prefect flow smoke.
- ☐ Каждый участник защищает собственный failure-сценарий.
- ☐ Итоговая версия отмечена tag lr9-v1.

25 Связь с ЛР10

В ЛР10 успешная модель из flow будет регистрироваться в MLflow Model Registry. Quality gate станет условием присвоения alias, а release manifest свяжет flow run, MLflow run, model version и решение о rollback.

26 Официальные источники

- Prefect documentation: https://docs.prefect.io/v3/
- Prefect flows and tasks: https://docs.prefect.io/latest/tutorial/flows
- Prefect package releases: https://pypi.org/project/prefect/
- Prefect source repository: https://github.com/PrefectHQ/prefect
- Python argparse: https://docs.python.org/3/library/argparse.html

ЛР10. MLflow Registry и управляемый выпуск модели

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Продолжительность: 2 академических часа и самостоятельная работа
Предшествующая работа: ЛР9 «Оркестрация обучения и обработка отказов»

1. Цель

Создать воспроизводимый процесс регистрации и выпуска модели: связать успешный Prefect run с MLflow model version, применить aliases candidate и champion, зафиксировать version tags и release manifest, отклонить модель при нарушении gate и выполнить rollback без переобучения.

2. Результаты обучения

Каждый студент должен уметь:

- различать MLflow run, logged model, registered model и model version;
- регистрировать sklearn-модель с signature и input example;
- загружать exact version и версию по alias;

- использовать aliases и tags вместо deprecated stages;
- объяснять автоматический gate и ручной approval;
- связывать commit, data hash, run ID и model version;
- доказать, что rejected candidate не изменяет champion;
- выполнить и объяснить rollback;
- диагностировать ошибку Registry по exact identifiers.

3. Командный результат

Команда предоставляет tag lr10-v1 и:

1. локальный MLflow Tracking/Registry с database-backed backend;
2. registered model с уникальным именем команды;
3. минимум две model versions из разных run или параметров;
4. signature и безопасный input example у каждой принимаемой версии;
5. version tags происхождения и проверки;
6. aliases candidate и champion;
7. successful release manifest;
8. rejected gate report, не изменивший champion;
9. rollback manifest и подтверждение загрузки восстановленной версии;
10. тесты pure-логики manifest/gate и CI-проверку;
11. issue, небольшие PR, review и индивидуальные evidence cards.

4. Проверенная версия и установка

На дату актуализации используется MLflow 3.16.x; зависимость проекта ограничена диапазоном mlflow>=3.16,<4. Перед семестром преподаватель повторно проверяет стабильную версию.

```
python -m pip install -e ".[dev,tracking,orchestration]"
mlflow --version
```

Stages Staging, Production, Archived и API transition_model_version_stage() не применяются. Используйте aliases и tags.

5. Локальная архитектура

```
Prefect run-summary.json + model.joblib + prepared.csv
                    |
                    v
              quality gate
                    |
                    v
MLflow run -> logged model -> registered model version
                                  |
                   candidate --+--> champion
                                  |
                              release manifest
```

Для Registry нужен database-backed backend. В учебном варианте используйте SQLite:

```
mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlartifacts \
  --host 127.0.0.1 \
  --port 5000
```

В PowerShell обратный слеш замените обратным апострофом или выполните команду одной строкой. Файлы mlflow.db, mlartifacts/ и mlruns/ не добавляются в Git.

6. Именование

Registered model name:

<код-команды>-flight-delay-classifier

Пример: team-03-flight-delay-classifier. Имя не включает run ID, фамилию студента или слово final. Разные команды не используют общее имя.

7. Подготовка двух кандидатов

Выполните ЛР9 минимум дважды с разными безопасными параметрами, например n_estimators=60 и n_estimators=100. Для каждого запуска сохраните собственный каталог и model path.

```
python -m mlops_flight_delay.orchestration \
  --data data/flights_sample.csv \
  --contract data/contract.json \
  --run-dir reports/prefect/lr10-run-a \
  --model models/lr10-a.joblib \
  --n-estimators 60
```

Не копируйте один и тот же файл под двумя именами. В summary должны различаться параметры или подтверждённый источник обучения.

8. Регистрация candidate

Пример команды starter project:

```
python -m mlops_flight_delay.registry register \
  --tracking-uri http://127.0.0.1:5000 \
  --model-name team-03-flight-delay-classifier \
  --owner-team team-03 \
  --model models/lr10-a.joblib \
  --data reports/prefect/lr10-run-a/prepared.csv \
  --summary reports/prefect/lr10-run-a/run-summary.json \
  --source-commit <полный-Git-SHA> \
  --minimum-roc-auc 0.50 \
  --output reports/registry/lr10-candidate-a.json
```

Команда должна:

1. проверить passed=true и порог метрики в summary;
2. проверить hash подготовленных данных;
3. загрузить bundle и построить smoke prediction;
4. залогировать модель с signature и input example;
5. создать model version;
6. записать tags validation_status, source_commit, data_sha256, owner_team;
7. назначить alias candidate;
8. сохранить exact version и run ID в JSON.

9. Проверка Registry

В UI и через MlflowClient подтвердите:

- registered model name;
- version number;
- source run ID;

- signature;
- version tags;
- alias candidate;
- загрузку models:/<name>/<version>;
- загрузку models:/<name>@candidate;
- одинаковую resolved version для двух способов загрузки.

Скриншот допустим только как дополнение. Основное evidence — JSON, код и воспроизводимая команда.

10. Tags

Обязательные tags model version:

Key	Значение
validation_status	passed или failed
source_commit	полный Git SHA
data_sha256	hash prepared dataset
quality_report	относительный путь к report
owner_team	код команды
release_status	candidate, approved или rolled_back

Запрещены токены, пароли, персональные данные, полный dataset и локальные абсолютные пути.

11. Gate перед promotion

Candidate допускается к promotion, если одновременно:

- upstream summary имеет passed=true;
- test ROC AUC не ниже --minimum-roc-auc;
- data hash совпадает;
- model bundle загружается;
- signature создана;
- smoke prediction возвращает конечные вероятности в диапазоне [0, 1];
- source_commit имеет формат полного SHA;
- version tags читаются обратно из Registry;
- review выполнен участником, не являющимся автором candidate.

Изменение порога фиксируется в issue и PR.

12. Promotion

```
python -m mlops_flight_delay.registry promote \
  --tracking-uri http://127.0.0.1:5000 \
  --model-name team-03-flight-delay-classifier \
  --data reports/prefect/lr10-run-a/prepared.csv \
  --approved-by student-reviewer-login \
  --output reports/releases/lr10-release.json
```

Promotion получает version по alias candidate, читает текущий champion, проверяет validation_status=passed, переназначает champion и затем читает alias повторно. Manifest должен сохранить previous_champion_version, даже если первого champion ещё не было.

13. Release manifest

Обязательные поля:

```
{
  "schema_version": "1.0.0",
  "model_name": "team-03-flight-delay-classifier",
  "model_version": "2",
  "previous_champion_version": "1",
  "mlflow_run_id": "...",
  "source_commit": "...",
  "data_sha256": "...",
  "resolved_model_uri": "models:/team-03-flight-delay-classifier/2",
  "quality_gate": {"passed": true, "minimum_roc_auc": 0.5},
  "decision": {"status": "approved", "approved_by": "..."}
}
```

Сведения берутся из Registry и отчётов, а не вводятся повторно вручную.

14. Rejected candidate

Создайте обратимый негативный сценарий: --minimum-roc-auc 1.1 либо копию summary с passed=false. Ожидаемый результат:

- команда завершилась ненулевым exit code;
- создан failure report;
- новая версия не получила champion;
- текущий champion остался неизменным;
- причина содержит фактическую метрику и порог;
- повторный запуск с корректными входами возможен.

Нельзя имитировать отказ удалением общей базы, artifact store или версии другого участника.

15. Rollback

После успешного promotion выполните контролируемый откат на previous champion:

```
python -m mlops_flight_delay.registry rollback \
  --tracking-uri http://127.0.0.1:5000 \
  --model-name team-03-flight-delay-classifier \
  --target-version 1 \
  --data reports/prefect/lr10-run-a/prepared.csv \
  --reason "controlled-lr10-drill" \
  --approved-by student-operator-login \
  --output reports/releases/lr10-rollback.json
```

После alias update загрузите models:/<name>@champion, выполните smoke prediction и запишите resolved version. Отозванная version не удаляется.

16. Идемпотентность и конкуренция

Повторное чтение и проверка не должны создавать version. register создаёт новую version только для нового принятого артефакта. promote не создаёт version и не переобучает модель.

Для учебной работы promotion выполняется последовательно. Перед update команда повторно получает candidate и сравнивает его с reviewed version. Если alias изменился, операция прекращается и требует нового review.

17. Тестирование

Обязательные тесты:

1. manifest содержит все обязательные поля;

2. неверный commit SHA отклоняется;
3. passed=false отклоняется;
4. метрика ниже порога отклоняется;
5. hash несоответствие отклоняется;
6. promotion требует validation_status=passed;
7. previous champion сохраняется;
8. rollback требует существующую target version;
9. секретоподобные ключи не попадают в manifest;
10. integration smoke создаёт version в временной SQLite Registry.

Pure tests должны работать без поднятого HTTP server. Integration test можно маркировать отдельно.

18. CI

CI устанавливает [dev,tracking,orchestration], запускает pure tests и локальный registry smoke с отдельной временной SQLite database. Долговременные mlflow.db и artifacts не публикуются; в diagnostic artifact сохраняются только sanitized JSON reports.

19. Командные роли

Для команды 2–5 человек роли совмещаются или распределяются:

- владелец регистрации и signature;
- владелец gate и отрицательного сценария;
- владелец tags и manifest;
- release operator и rollback;
- reviewer происхождения и CI.

Роли должны отличаться от ЛР9. Координатор без технического или проверочного артефакта не считается отдельным вкладом.

20. Индивидуальная сдача участника команды

Командный результат: tag lr10-v1, registered model, versions, aliases, manifests и reports.

Каждый участник размещает отдельную evidence card. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР10 и tag lr10-v1
Роль	роль в ЛР10 и отличие от ЛР9
Задача	issue с наблюдаемым критерием
Личный артефакт	PR, test, version, tag policy, manifest, review или rollback report
Exact identifiers	commit SHA, run ID, model name и version
Решение	почему выбран alias/tag/gate и рассмотренная альтернатива
Риск	stale candidate, неверный hash, broken artifact, секрет или конфликт promotion
Отказ	личный rejected/promotion/rollback сценарий и ожидаемое состояние champion
Использование ИИ	инструмент, самостоятельно проверенные утверждения или отсутствие использования

Минимальный допуск

1. Командная версия зафиксирована неизменяемым tag.

2. Участнику назначена собственная issue.
3. Есть содержательный личный артефакт; число commit не является доказательством.
4. Изменение прошло PR и review другого участника.
5. Evidence содержит рабочие exact identifiers и ссылки.
6. Автоматические проверки зелёные либо исключение письменно принято преподавателем.
7. Участник не является одновременно единственным автором и единственным approving reviewer своего promotion.

21. Индивидуальная защита

За 3–4 минуты участник:

1. показывает личный артефакт в tag lr10-v1;
2. называет model name, exact version, run ID и commit;
3. загружает exact version или версию по alias;
4. показывает signature, tags и report;
5. объясняет различие candidate и champion;
6. воспроизводит назначенный rejected или rollback-сценарий;
7. доказывает, что champion изменился или остался прежним согласно сценарию;
8. объясняет один риск и способ обнаружения.

Преподаватель может изменить порог, candidate alias, target rollback или вход smoke prediction.

22. Повторная сдача и стоп-факторы

Повторная сдача выполняется через issue, отдельный PR, новый report и обновлённую evidence card. Первая попытка не удаляется.

Стоп-факторы:

- использование deprecated stages как основного механизма;
- отсутствие signature или source run;
- alias назначен версии с failed gate;
- manifest заполнен вручную и противоречит Registry;
- champion переназначен без review/approval;
- rollback переобучает модель или удаляет историю;
- один общий failure-сценарий выдан всем участникам;
- в репозитории оказались токены, база с чужими данными или персональные сведения;
- скриншот заменяет код, exact identifiers и reports.

23. Критерии оценивания

Критерий	Баллы	Полное выполнение
Registration и signature	2	две versions связаны с run и загружаются
Gate и происхождение	2	commit, data hash, метрики и evidence проверяются
Aliases, tags и promotion	2	candidate/champion применены без stages
Manifest и rollback	2	previous champion сохранён и откат воспроизводим
Индивидуальная защита	2	личный артефакт, отказ, диагностика и решение

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

24. Чек-лист команды

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ MLflow 3 используется из проверенного диапазона.
- ☐ Backend Registry основан на SQLite или другой database-backed store.
- ☐ Зарегистрированы минимум две реальные model versions.
- ☐ У model versions есть signature и source run.
- ☐ Aliases candidate и champion разрешаются в exact versions.
- ☐ Tags происхождения читаются обратно из Registry.
- ☐ Rejected candidate не изменяет champion.
- ☐ Release manifest сформирован из фактических данных.
- ☐ Rollback возвращает предыдущую version без retraining.
- ☐ Smoke prediction выполнен после promotion и rollback.
- ☐ CI и тесты успешны.
- ☐ У каждого участника собственная evidence card и failure-сценарий.
- ☐ Итоговая версия отмечена tag lr10-v1.

25. Связь с ЛР11

В ЛР11 сервис будет публиковать resolved model name/version в структурированном логе и метриках. Наблюдаемость должна позволять сравнить поведение разных выпусков и связать деградацию с release manifest.

26. Официальные источники

- MLflow Model Registry workflow: https://mlflow.org/docs/latest/ml/model-registry/workflow/
- MLflow Registry tutorial: https://mlflow.org/docs/latest/ml/model-registry/tutorial
- MLflow Client API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.client.html
- MLflow package releases: https://pypi.org/project/mlflow/

Часть III. Эксплуатация, governance и итоговый выпуск

ЛР11. Наблюдаемость ML-сервиса

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Продолжительность: 2 академических часа и самостоятельная работа
Предшествующая работа: ЛР10 «MLflow Registry и управляемый выпуск модели»

1. Цель

Создать воспроизводимый контур наблюдаемости ML-сервиса: ограниченные HTTP-метрики, JSON-логи, Prometheus scrape и rules, provisioned Grafana dashboard, формализованные SLI/SLO, контролируемый alert-сценарий и runbook, связывающий запрос с model name/version и release evidence.

2. Результаты обучения

Каждый студент должен уметь:

- различать metrics, logs, traces и evidence;
- выбирать Counter, Gauge и Histogram по смыслу;
- ограничивать cardinality labels;
- вычислять rate, error ratio и p95 через PromQL;

- объяснять bucket, окно и counter reset;
- находить JSON-log по request ID;
- связывать запрос с resolved model version;
- формулировать SLI/SLO с population и окном;
- переводить alert через pending, firing и resolved;
- применять runbook и подтверждать восстановление.

3. Командный результат

Команда предоставляет tag lr11-v1 и:

1. FastAPI service с /healthz, /readyz, /predict, /metrics;
2. HTTP counter и latency histogram с ограниченными labels;
3. prediction outcome metric и model_info;
4. JSON-log запроса с request ID, status, duration и model identity;
5. Compose stack app + Prometheus + Grafana;
6. version-controlled Prometheus config и rules;
7. provisioned datasource и dashboard минимум из шести панелей;
8. SLI/SLO document;
9. минимум один alert и проверяемый runbook;
10. общий baseline load report и индивидуальные load/incident reports;
11. tests и CI validation конфигурации;
12. issue, небольшие PR, review и индивидуальные evidence cards.

4. Проверенные версии

На 20 октября 2026 года starter project использует:

- prometheus-client>=0.26,<1;
- Prometheus v3.15.0;
- Grafana OSS 13.2.4.

Перед семестром преподаватель проверяет stable releases и security notes. Плавающие image tags latest не применяются.

5. Запуск

```
cp .env.example .env
## задайте случайный локальный GRAFANA_ADMIN_PASSWORD и полный SOURCE_COMMIT
docker compose up --build --detach
docker compose ps
```

Локальные адреса:

- API: http://127.0.0.1:8080;
- Prometheus: http://127.0.0.1:9090;
- Grafana: http://127.0.0.1:3000.

Пароль не добавляется в Git. Порты привязаны к loopback; публикация учебного Grafana в интернет не требуется.

6. Проверка targets и provisioning

В Prometheus target flight-delay-api должен иметь состояние UP. В Grafana автоматически появляются datasource Prometheus, папка MLOps и dashboard Flight Delay Service.

Если dashboard создан только вручную, работа не принимается: source JSON и provisioning YAML должны находиться в tag lr11-v1.

7. Обязательные метрики

Метрика	Тип	Labels	Назначение
http_requests_total	Counter	method, route, status	request rate и errors
http_request_duration_seconds	Histogram	method, route	latency distribution и p95
prediction_outcomes_total	Counter	outcome, model_version	результат классификации и ошибки
model_info	Gauge	model_name, model_version, source_commit	identity активной модели
up	Gauge Prometheus	job, instance	доступность scrape target

Запрещённые labels: request ID, student ID, user ID, полный URL, timestamp, текст ошибки, входные признаки и probability.

8. Route template

Label route принимает шаблон /predict, /readyz, /metrics или значение unmatched. Не используйте сырой path с идентификаторами. Test должен доказать, что разные request ID не создают разные series.

9. JSON-лог

Для каждого HTTP-request сервис пишет одну JSON-строку:

```
{
  "event": "http_request",
  "request_id": "lr11-student-01-0001",
  "method": "POST",
  "route": "/predict",
  "status": 200,
  "duration_ms": 8.7,
  "model_name": "flight-delay-classifier",
  "model_version": "3"
}
```

Лог не содержит request body, признаки, токены, пароль Grafana и персональные данные. При ошибке сохраняется тот же request ID, который возвращён в header.

10. Связь с выпуском ЛР10

В .env задайте MODEL_NAME, MODEL_VERSION и SOURCE_COMMIT из принятого release manifest. Подтвердите одинаковые значения:

1. в manifest ЛР10;
2. в model_info;
3. в JSON-log;
4. в response /predict для model version.

Если локальный starter bundle используется без Registry, укажите MODEL_VERSION=local и объясните ограничение; для полного балла применяется exact version ЛР10.

11. Воспроизводимая нагрузка

Командный baseline:

```
python scripts/observability_load.py \
  --base-url http://127.0.0.1:8080 \
  --requests 200 \
  --invalid-every 10 \
  --student-id team-baseline \
  --output reports/observability/lr11-team-baseline.json
```

Сценарий создаёт ожидаемые 200 и 422, но не 5xx. Report содержит число запросов, статусы, mean, p50, p95 и max. Он не заменяет Prometheus: локальные client timings и server histogram сравниваются как разные точки измерения.

12. Индивидуальная нагрузка

Каждый участник выбирает собственные --requests, --invalid-every и --student-id, фиксирует ожидания до запуска и сохраняет отдельный report. Запуск всей команды под одним ID не принимается.

Нагрузка выполняется последовательно и ограниченно. Цель — получить наблюдаемый сигнал, а не провести стресс-тест общей машины.

13. PromQL: request rate

```
sum by (route) (rate(http_requests_total[5m]))
```

Evidence содержит query, окно, время выполнения, result и объяснение. Студент должен показать, почему rate применяется до sum.

14. PromQL: error ratio

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
clamp_min(sum(rate(http_requests_total[5m])), 0.001)
```

422 анализируются отдельным query:

```
sum(rate(http_requests_total{status="422"}[5m]))
```

Не называйте 422 server failure без обоснования.

15. PromQL: p95

```
histogram_quantile(
  0.95,
  sum by (route, le) (rate(http_request_duration_seconds_bucket[5m]))
)
```

Label le обязателен для classic histogram. Студент объясняет, почему p95 зависит от buckets и не равен максимуму.

16. Dashboard

Обязательные панели:

1. request rate;
2. 5xx ratio;
3. p95 latency by route;
4. prediction outcomes by model version;
5. target up;
6. resolved model_info.

У каждой панели есть title, query, единица, окно и понятная legend. Dashboard JSON проходит parse и хранится в Git. UI-изменения экспортируются обратно в source либо не учитываются.

17. SLI/SLO

Создайте docs/observability/sli-slo.md минимум с двумя SLI:

Availability SLI = valid /predict requests without 5xx / all valid /predict requests.
Latency SLI = fraction of valid /predict requests completed within 500 ms.

Для каждого SLO укажите population, good/bad event, target, окно, минимум трафика, PromQL и ограничение измерения. Учебный пример: availability >=99% и p95 <=500 ms за 15 минут при минимум 50 запросах.

18. Alert rules

Starter содержит:

- FlightDelayServiceDown: up==0, for: 1m;
- FlightDelayHigh5xxRatio: ratio >5%, for: 2m;
- FlightDelayHighP95Latency: p95 >500 ms, for: 2m.

Команда сохраняет один общий baseline; каждый студент отвечает за один alert или отдельную модификацию порога/окна. Alert должен иметь severity, team, summary и runbook annotation.

19. Контролируемые сценарии

Допустимые индивидуальные сценарии:

Сценарий	Ожидаемый сигнал	Восстановление
остановить только app	up=0, Down pending/firing	запустить app, дождаться resolved
временный тестовый 500 endpoint в отдельной ветке	5xx ratio	отменить commit/ветку
контролируемая задержка в отдельной ветке	p95 alert	убрать задержку
неверный model path	/readyz=503, not-ready outcomes	вернуть путь и рестарт
Prometheus не видит target	scrape failure	исправить config/network

Нельзя удалять чужие volumes, портить общий dashboard, бесконечно генерировать нагрузку или публиковать сервис наружу.

20. Alert lifecycle

Для своего сценария участник фиксирует:

1. baseline и release identity;
2. момент введения отказа;
3. pending;
4. firing;
5. выполненные шаги runbook;
6. восстановление;
7. resolved;
8. краткую причину и профилактику.

Если семинар ограничен по времени, преподаватель может уменьшить for в отдельной учебной ветке; изменение фиксируется и не попадает молча в tag.

21. Runbook

docs/runbooks/flight-delay-api.md должен содержать область, первые проверки, диагностику каждого alert, безопасные действия и критерий завершения. Команда дополняет runbook конкретными командами и владельцами.

Фраза «перезапустить всё» не считается runbook. Перезапуск допустим только после фиксации evidence и объяснения гипотезы.

22. Тестирование и CI

Обязательные проверки:

1. /metrics содержит обязательные метрики;
2. успешный /predict создаёт series с route template и status 200;
3. invalid request создаёт status 422 без request ID label;
4. JSON-log parseable и содержит model identity;
5. JSON-log не содержит payload;
6. dashboard JSON parseable;
7. Compose configuration валидна;
8. promtool check config проверяет config и rules;
9. image tags Prometheus/Grafana не плавающие;
10. .env не отслеживается Git.

CI не обязан поднимать долгоживущий Grafana. Он проверяет tests, Compose, Prometheus config/rules и сборку service image.

23. Командные роли

Для команды 2–5 человек:

- владелец application metrics и cardinality;
- владелец JSON logs/correlation/privacy;
- владелец PromQL и SLI/SLO;
- владелец Grafana provisioning/dashboard;
- владелец alerts/runbook/incident drill.

В малой команде роли совмещаются, но артефакты различаются. Роли ротируются относительно ЛР10.

24. Индивидуальная сдача участника команды

Командный результат: tag lr11-v1, observability stack, dashboard, rules, runbook и baseline report.

Каждый участник заполняет собственную evidence card. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Лабораторная	ЛР11 и tag lr11-v1
Роль	роль в ЛР11 и отличие от ЛР10
Задача	issue с наблюдаемым критерием
Личный артефакт	metric, query, panel, log schema, alert, runbook или test
Нагрузка	команда, параметры и личный JSON-report
Evidence	PromQL, окно, результат, request ID и model version
Отказ	личный сценарий, pending/firing/resolved и восстановление

Поле	Что указать
Решение	labels/buckets/window/threshold и рассмотренная альтернатива
Риск	cardinality, PII/secrets, alert fatigue или неверный SLI
Использование ИИ	инструмент и самостоятельно проверенные утверждения

Минимальный допуск

1. Командная версия зафиксирована tag lr11-v1.
2. Участнику назначена собственная issue.
3. Есть содержательный личный артефакт и отдельный report.
4. Изменение прошло PR и review другого участника.
5. Evidence содержит query, окно, request ID и model identity.
6. Автоматические проверки успешны либо исключение письменно принято преподавателем.
7. Секреты, request bodies и персональные данные отсутствуют.

25. Индивидуальная защита

За 3–4 минуты участник:

1. показывает личный артефакт в tag;
2. запускает свой профиль нагрузки;
3. выполняет назначенный PromQL query;
4. находит JSON-log по request ID;
5. связывает log и model_info с release manifest;
6. объясняет labels, окно и тип метрики;
7. воспроизводит отказ или разбирает сохранённый lifecycle;
8. выполняет шаги runbook и подтверждает восстановление.

Преподаватель может изменить окно, threshold, долю invalid requests или попросить классифицировать новый label как безопасный/опасный.

26. Повторная сдача и стоп-факторы

Повторная сдача выполняется через issue, отдельный PR, новый load/incident report и обновлённую evidence card. История первой попытки сохраняется.

Стоп-факторы:

- request ID, user ID или raw path используется как label;
- dashboard создан только вручную;
- лог содержит признаки, пароль, токен или персональные данные;
- p95 вычисляется без histogram buckets или le;
- alert не имеет окна/владельца/runbook;
- общий скриншот заменяет query и report;
- model version не связана с выпуском;
- один incident report выдан всей команде;
- отказ разрушает общую среду или evidence;
- студент не может объяснить собственный label/query/threshold.

27. Критерии оценивания

Критерий	Баллы	Полное выполнение
Метрики и cardinality	2	корректные типы, names, labels и tests
Logs и release correlation	2	JSON, request ID, model identity, privacy
PromQL, dashboard и SLI/SLO	2	queries корректны, provisioning воспроизводим
Alert и runbook	2	lifecycle доказан, восстановление проверено
Индивидуальная защита	2	личная нагрузка, артефакт, диагностика и решение

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

28. Чек-лист

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ Versions инструментов проверены и image tags фиксированы.
- ☐ Prometheus target имеет состояние UP.
- ☐ Dashboard и datasource provisioned из Git.
- ☐ Metrics используют route template и ограниченные labels.
- ☐ JSON-log parseable и не содержит payload/secrets.
- ☐ Release identity согласована с ЛР10.
- ☐ Baseline и личные load reports сохранены.
- ☐ Rate, 5xx ratio и p95 PromQL воспроизводимы.
- ☐ SLI/SLO содержат population, target и окно.
- ☐ Alert прошёл pending/firing/resolved или сохранён эквивалентный evidence.
- ☐ Runbook завершился проверкой восстановления.
- ☐ CI и tests успешны.
- ☐ У каждого участника отдельная evidence card.
- ☐ Итоговая версия отмечена tag lr11-v1.

29. Связь с ЛР12

В ЛР12 временные окна, model version и release manifest будут использованы для мониторинга drift, delayed labels, coverage и качества. Service health останется отдельным слоем: доступный API может обслуживать деградировавшую модель, а качественная модель может быть недоступна из-за отказа сервиса.

30. Официальные источники

- Prometheus instrumentation practices: https://prometheus.io/docs/practices/instrumentation/
- Prometheus query functions: https://prometheus.io/docs/prometheus/latest/querying/functions/
- Prometheus alerting rules: https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/
- Grafana provisioning: https://grafana.com/docs/grafana/latest/administration/provisioning/
- Grafana Docker installation: https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/
- Python Prometheus client releases: https://pypi.org/project/prometheus-client/

ЛР12. Drift и отложенное качество модели

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей

Продолжительность: 2 академических часа и самостоятельная работа
Предшествующая работа: ЛР11 «Наблюдаемость ML-сервиса»

1. Цель

Построить воспроизводимый мониторинг данных и качества модели: сравнить reference/current окна, сформировать drift report, корректно присоединить задержанные labels, оценить coverage и качество, а затем принять управляемое решение по version-controlled policy.

2. Результаты обучения

Каждый студент должен уметь:

- различить drift, data quality incident и quality degradation;
- объяснить границы reference/current окна;
- воспроизвести PSI и один его риск;
- рассчитать mature cohort и label coverage;
- связать запись с model version и release manifest;
- интерпретировать ROC AUC/F1 только при достаточных labels;
- выполнить отказной сценарий и применить runbook;
- обосновать hold, investigate, incident_review или retraining_review.

3. Командный результат

Команда предоставляет tag lr12-v1 со следующими артефактами:

1. monitoring/drift_policy.json;
2. модуль расчёта drift и delayed quality;
3. reference/current datasets с явными временными границами;
4. prediction log и label log с уникальным request_id;
5. JSON-отчёт с PSI, coverage, metrics и decision;
6. минимум четыре автоматических теста;
7. runbook реакции;
8. один общий baseline report и отдельный отчёт каждого участника;
9. issue, небольшие PR, review и индивидуальные evidence cards.

4. Проверенная версия расширения

Обязательный расчёт использует pandas и NumPy из starter-project. Дополнительно разрешён Evidently >=0.7.23,<0.8, проверенный 20 октября 2026 года. Если команда использует Evidently, она сохраняет точную версию, конфигурацию, JSON и объясняет различия с прозрачным baseline. Один HTML без policy и машинно-читаемого результата не принимается.

5. Подготовка

```
python -m pip install -e ".[dev]"
python -m pytest tests/test_monitoring.py -q
```

Для дополнительного Evidently report:

```
python -m pip install -e ".[monitoring]"
```

Не публикуйте production-данные, персональные сведения, токены и сырые чувствительные признаки.

6. Входные контракты

Reference/current содержат DepHour и IsWeekend. Prediction log содержит:

Поле	Требование
request_id	уникальный идентификатор
event_time	UTC timestamp прогноза
prediction_probability	число от 0 до 1
model_version	точная версия из release manifest
признаки	только разрешённые policy поля

Label log содержит request_id, label_time, ArrDelay. Цель определяется как ArrDelay >= 15. Join должен быть one-to-one.

7. Reference и current окна

Команда документирует источник, начало, конец, фильтры и число строк обоих окон. Reference должен предшествовать current либо соответствовать утверждённой validation-выборке champion. Запрещено подбирать reference после просмотра current только ради прохождения threshold.

8. Проверка качества данных до drift

До статистики проверьте схему, типы, пропуски, диапазон DepHour, бинарность IsWeekend, уникальность идентификаторов и timestamps. Нарушение контракта классифицируется отдельно. Нельзя маскировать значение 900 широкими bins и называть это естественным drift.

9. Числовой PSI

Для DepHour постройте quantile bins только по reference. Примените те же границы к current, сгладьте нулевые доли и сохраните score. В отчёте укажите method psi_reference_bins, warning, alert и число строк. Студент должен уметь объяснить влияние числа bins.

10. Категориальный PSI

Для IsWeekend используйте объединение категорий reference и current. Новая категория не должна исчезать из расчёта. Сохраните score и статус. Для реального высококардинального признака команда обязана согласовать агрегацию редких категорий.

11. Policy

Минимальная policy содержит:

```
{
  "minimum_window_rows": 8,
  "psi_warning": 0.1,
  "psi_alert": 0.25,
  "expected_label_delay_hours": 24,
  "minimum_labeled_rows": 8,
  "minimum_label_coverage": 0.75,
  "minimum_roc_auc": 0.7,
  "minimum_f1": 0.6,
  "automatic_retraining": false
}
```

Учебные значения можно изменить до запуска с письменным обоснованием. Подгонять пороги после получения результата запрещено.

12. Базовый запуск

```
python -m mlops_flight_delay.monitoring \
  --reference data/monitoring/reference.csv \
  --current data/monitoring/current.csv \
  --predictions data/monitoring/predictions.csv \
  --labels data/monitoring/labels.csv \
```

```
  --policy monitoring/drift_policy.json \
  --as-of 2026-10-20T12:00:00Z \
  --output reports/monitoring/lr12-team-baseline.json
```

Отчёт должен воспроизводиться из tag без ручного редактирования результата.

13. Mature cohort

В coverage включаются прогнозы с event_time <= as_of - expected_label_delay. Свежие события исключаются из знаменателя. В отчёте обязательны as_of, eligible_cutoff, число eligible и число matched labels. Все времена должны содержать часовой пояс.

14. Coverage gate

Если matched rows меньше minimum_labeled_rows или coverage ниже порога, статус — insufficient_evidence. Метрики качества не используются для решения о retraining. Команда должна показать, что отсутствие метки и отрицательная метка — разные состояния.

15. Метрики качества

При достаточных labels рассчитайте ROC AUC, precision, recall и F1 по одной model version. Если в окне один класс, ROC AUC не подменяется нулём: отчёт получает insufficient_evidence. Threshold классификации фиксируется до расчёта.

16. Сегментный анализ

Добавьте минимум один безопасный разрез, например IsWeekend. Для каждого сегмента укажите N и coverage. Не делайте вывод по малому сегменту. Индивидуальный участник может отвечать за отдельный сегмент, но должен использовать общую policy и командный release.

17. Decision gate

Starter-project реализует следующие решения:

Условие	Действие
labels недостаточно	hold
drift alert, качество в норме	investigate
quality breach без drift alert	incident_review
drift alert и quality breach	retraining_review
пороги не нарушены	no_action

retraining_review создаёт процесс проверки candidate, но не назначает его champion автоматически.

18. Отказные сценарии

Каждый участник выбирает собственный сценарий:

Сценарий	Ожидаемый результат
удалить зрелые labels	coverage падает, решение hold
дублировать request_id	явная ошибка one-to-one контракта
сдвинуть as_of	изменяется mature cohort
добавить недопустимый DepHour	data quality incident до drift
оставить один класс	ROC AUC не рассчитывается
изменить только current distribution	investigate, а не auto retraining

Работайте в отдельной ветке или копии входных данных. Общий baseline не портится.

19. Runbook

Используйте docs/runbooks/model-monitoring.md. Зафиксируйте первые проверки, владельца, действие, условие завершения и ссылку на evidence. Команда не должна ограничиваться фразой «переобучить».

20. Postmortem

Для одного общего сценария создайте краткий postmortem: временная линия, влияние, первопричина, что помогло обнаружить, что задержало реакцию, corrective action и владелец. Учебный инцидент явно маркируется как моделирование.

21. Автоматические тесты

Обязательны проверки:

1. drift обнаруживается на изменённом current;
2. bins формируются без использования current;
3. свежие прогнозы исключаются из coverage;
4. дубликат request_id отклоняется;
5. insufficient labels дают hold;
6. retraining review требует одновременно drift и quality breach;
7. JSON parseable и содержит policy version;
8. линтер и весь pytest проходят.

22. CI

Workflow запускает тесты и базовую команду мониторинга, сохраняет JSON в diagnostic artifacts. Evidently UI в CI поднимать не требуется. Floating dependency без upper bound не принимается.

23. Командные роли

Для команды 2–5 человек:

- владелец окон и data quality;
- владелец drift metric и статистических ограничений;
- владелец delayed-label join и coverage;
- владелец quality gate и тестов;
- владелец runbook/postmortem и release correlation.

В малой команде роли совмещаются, но артефакты различаются. Роли ротируются относительно ЛР11.

24. Индивидуальная сдача участника команды

Командный результат: tag lr12-v1, policy, тесты, baseline JSON, runbook и postmortem.

Каждый участник отдельно заполняет docs/evidence/lr12-evidence-card.md. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда и GitHub-логин
Лабораторная	ЛР12 и tag lr12-v1
Роль	роль и отличие от роли в ЛР11
Задача	собственная issue с наблюдаемым критерием
Личный артефакт	metric, window, join, test, policy, report, runbook или postmortem
Запуск	личные параметры, as_of и отдельный JSON

Поле	Что указать
Evidence	score, coverage, metric, decision и model version
Решение	альтернатива, риск и проверяемое обоснование
Использование ИИ	инструмент и самостоятельно проверенные утверждения

25. Минимальный допуск

1. Командная версия зафиксирована tag и commit SHA.
2. Участнику назначена собственная issue.
3. Есть содержательный личный артефакт и отдельный JSON-отчёт.
4. Изменение прошло PR и review другого участника.
5. Evidence card содержит рабочие ссылки.
6. CI успешен либо исключение письменно принято преподавателем.
7. Студент может объяснить окно, знаменатель coverage и принятое действие.

26. Индивидуальная защита

За 3–4 минуты участник:

1. показывает свой артефакт в tag;
2. воспроизводит личный сценарий;
3. объясняет reference/current и mature cohort;
4. находит model version и policy version в отчёте;
5. интерпретирует один PSI и одну quality metric;
6. выполняет выбранное преподавателем изменение as_of, threshold или входного файла;
7. объясняет, почему решение не означает автоматический выпуск.

27. Повторная сдача и стоп-факторы

Повторная сдача выполняется через переоткрытую issue, новый PR, обновлённый отчёт и evidence card. История первой попытки сохраняется.

Стоп-факторы:

- current использован для построения reference bins;
- свежие прогнозы ошибочно включены как missing labels;
- метрики рассчитаны при недостаточном coverage;
- model version или временные границы отсутствуют;
- drift автоматически запускает promotion;
- HTML заменяет policy и JSON;
- общий отчёт выдан за личный;
- студент не может объяснить собственное решение.

28. Критерии оценивания

Критерий	Баллы	Полное выполнение
Окна, контракт и drift	2	воспроизводимые границы, корректный PSI и ограничения
Delayed labels и coverage	2	mature cohort, one-to-one join и достаточность evidence
Quality и decision gate	2	метрики корректны, auto retraining отсутствует

Критерий	Баллы	Полное выполнение
Тесты, CI и runbook	2	отказные сценарии воспроизводимы, реакция завершена
Индивидуальный вклад и защита	2	личный отчёт, артефакт, review и объяснение

Максимум: 10 баллов. Рекомендуемый порог зачёта — 6 баллов при отсутствии стоп-факторов.

29. Чек-лист

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ Reference/current границы и model version зафиксированы.
- ☐ Data quality проверена до drift.
- ☐ Числовые bins построены только по reference.
- ☐ Mature cohort использует expected label delay.
- ☐ Coverage содержит числитель и знаменатель.
- ☐ Quality не считается при недостаточных labels.
- ☐ Decision соответствует policy.
- ☐ Автоматическое promotion отсутствует.
- ☐ Runbook и postmortem содержат владельцев.
- ☐ Тесты и CI успешны.
- ☐ У каждого участника отдельный отчёт и evidence card.
- ☐ Итоговая версия отмечена tag lr12-v1.

30. Связь с ЛР13

В ЛР13 временная корректность распространяется на признаки: студент сравнит наивный join с point-in-time join. Ошибка available time способна создать leakage в обучении и исказить drift, поэтому timestamps и контракт ЛР12 сохраняются.

31. Официальные источники

- Evidently PyPI: https://pypi.org/project/evidently/
- Evidently API Report и DataDriftPreset: https://evidentlyai.github.io/evidently/api-reference/v0.7.19/evidently.html
- Evidently presets API: https://evidentlyai.github.io/evidently/api-reference/main/evidently/presets.html
- scikit-learn classification metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html

ЛР13. Point-in-time признаки и Feature Store

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Продолжительность: 2 академических часа и самостоятельная работа
Предшествующая работа: ЛР12 «Drift и отложенное качество модели»

1. Цель

Сформировать исторически корректную выборку признаков, доказать утечку в наивном latest join, учесть available time и TTL, а затем связать contract с демонстрационным Feast repository.

2. Результаты обучения

Каждый студент должен уметь:

- объяснить entity, event time, feature time и available time;

- воспроизвести наивный и point-in-time join;
- доказать отсутствие будущих значений;
- интерпретировать feature age и coverage;
- диагностировать нарушение TTL или ключа;
- объяснить offline и online retrieval;
- связать feature service с версией модели;
- защитить личный тест, contract или audit.

3. Командный результат

Команда предоставляет tag lr13-v1:

1. features/feature_contract.json;
2. entity events и feature history с UTC timestamps;
3. naive.csv и point-in-time.csv;
4. audit.json с leakage, coverage и feature age;
5. минимум четыре автоматических теста;
6. определения Feast как необязательное расширение;
7. документацию временной семантики;
8. общий baseline и отдельный audit каждого участника;
9. issue, PR, review и индивидуальные evidence cards.

4. Проверенная версия расширения

Обязательная реализация использует pandas. Feast допускается для демонстрации в версии >=0.67,<0.68; stable 0.67.0 проверена 20 октября 2026 года. Установка Feast и запуск online store не являются условием допуска. Скриншот UI не заменяет definitions, тест и audit JSON.

5. Подготовка

```
python -m pip install -e ".[dev]"
python -m pytest tests/test_features.py -q
```

Дополнительное Feast-окружение:

```
python -m pip install -e ".[feature_store]"
```

6. Учебная схема

Entity events:

Поле	Смысл
request_id	уникальная строка решения
airport_id	entity join key
event_time	момент, для которого строится прогноз
ArrDelay	отложенная цель, не признак

Feature history содержит airport_id, feature_time, available_time, trailing_delay_rate, flights_last_24h.

7. Feature contract

Contract фиксирует version, entity key, три timestamp-семантики, список признаков, TTL, допустимую долю пропусков и владельца. Изменение units или смысла признака требует новой версии.

8. Проверки до join

Проверьте обязательные столбцы, UTC timestamps, уникальность request_id, тип entity key и временную идентичность feature history. Недетерминированные дубликаты отклоняются до join.

9. Наивный baseline

naive_latest_join выбирает последнюю строку истории для каждого аэропорта без учёта event time. Это намеренно ошибочный пример. Сохраните его результат и подсчитайте строки, где feature time или available time позже события.

10. Point-in-time join

Для каждого события оставьте feature rows той же entity, которые удовлетворяют:

- feature_time <= event_time;
- available_time <= event_time;
- возраст неотрицателен и не превышает TTL.

Выберите последнюю допустимую строку. Результат содержит одну строку на request_id.

11. Available time fixture

В истории аэропорта A запись относится к 07:00, но доступна только в 09:00. Для события в 08:00 корректный join обязан выбрать более раннее значение 0.35, а не 0.55. Этот fixture обязателен: он доказывает, почему одного feature time недостаточно.

12. TTL

Базовый TTL равен 48 часам. Измените его в личной ветке и покажите влияние на coverage. Нельзя увеличивать TTL только ради заполнения всех строк без анализа допустимой устарелости.

13. Базовый запуск

```
python -m mlops_flight_delay.features \
  --entities data/features/entities.csv \
  --history data/features/history.csv \
  --contract features/feature_contract.json \
  --output-dir reports/features/lr13-team-baseline
```

Ожидаемый audit: шесть entity rows, leakage в naive join, ноль leakage в point-in-time join, coverage 1.0 и status pass.

14. Audit JSON

Отчёт содержит contract version, число строк, naive_leakage_rows, point_in_time_leakage_rows, rows_changed_vs_naive, coverage, maximum feature age и status. Команда не удаляет наивный результат: он является проверяемым отрицательным примером.

15. Coverage

Coverage равен доле событий, для которых найдены все обязательные признаки. Укажите числитель и знаменатель в пояснении. При пропуске исследуйте entity mismatch, TTL, available time и источник. Молчаливое заполнение средним не принимается.

16. Feature age

Для выбранной строки сохраните возраст в часах. Отрицательный возраст означает leakage, возраст выше TTL — stale feature. Максимум полезен как evidence, но распределение возрастов информативнее одного числа; участник может добавить histogram как личный артефакт.

17. Training-serving parity

Зафиксируйте имя, тип, units и transformation каждого признака. Покажите, что inference ожидает тот же набор и contract version. Если online value сравнивается с historical, укажите сопоставимый момент и допустимую freshness.

18. Feast demo

Каталог feature_repo/ содержит entity airport, feature view airport_stats и feature service flight_delay_v1. Перед feast apply преобразуйте CSV в Parquet. Для historical retrieval передайте entity dataframe с airport_id и timestamp. Сравните результат с pandas point-in-time output.

19. Historical и online retrieval

get_historical_features используется для training и batch scoring. get_online_features получает актуальные значения для inference и не строит историческую выборку. На защите студент объясняет это различие даже если Feast не устанавливался.

20. Отказные сценарии

Каждый участник выбирает собственный сценарий:

Сценарий	Ожидаемый результат
убрать фильтр feature time	audit обнаруживает будущее
убрать available time	fixture в 08:00 выбирается неверно
уменьшить TTL	coverage падает
дублировать request ID	явный отказ контракта
изменить entity key	появляются пропуски или неверный join
использовать online latest для training	расхождение с historical output

Сценарий выполняется в отдельной ветке или копии данных.

21. Автоматические тесты

Обязательны проверки:

1. feature time не позже event time;
2. available time не позже event time;
3. fixture недоступной записи выбирает прошлое значение;
4. naive join содержит leakage;
5. point-in-time join не содержит leakage;
6. duplicate request ID отклоняется;
7. feature age не превышает TTL;
8. JSON parseable и содержит contract version.

22. CI

Workflow запускает весь pytest и CLI-аудит, сохраняет reports/features/lr13-ci. Feast не устанавливается в основной job, потому что является расширением. Команда может добавить отдельный manual job с pinned extra, не замедляя обязательный quality gate.

23. Командные роли

Для команды 2–5 человек:

- владелец entity contract и качества входов;
- владелец point-in-time алгоритма;
- владелец leakage fixtures и тестов;

- владелец parity/coverage audit;
- владелец Feast definitions и документации.

В паре роли совмещаются, но у каждого остаётся самостоятельный артефакт. Роли ротируются после ЛР12.

24. Индивидуальная сдача участника команды

Командный результат: tag lr13-v1, contract, два join output, audit, тесты и документация.

Каждый участник заполняет docs/evidence/lr13-evidence-card.md. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда и GitHub-логин
Лабораторная	ЛР13 и tag lr13-v1
Роль	роль и отличие от ЛР12
Задача	собственная issue с критерием приёмки
Личный артефакт	join, contract, test, fixture, audit, parity или Feast definition
Запуск	личный сценарий и отдельный output directory
Evidence	leakage rows, coverage, feature age и contract version
Решение	временная семантика, альтернатива и риск
Использование ИИ	инструмент и самостоятельно проверенные утверждения

25. Минимальный допуск

1. Командная версия зафиксирована tag и commit SHA.
2. Есть собственная issue и наблюдаемый критерий.
3. Личный артефакт встроен в командную версию.
4. Изменение прошло PR и review другого участника.
5. Личный audit воспроизводим.
6. CI успешен либо исключение письменно принято преподавателем.
7. Студент объясняет все три времени и TTL.

26. Индивидуальная защита

За 3–4 минуты участник:

1. показывает личный артефакт в tag;
2. воспроизводит свой отказной сценарий;
3. находит leakage в naive output;
4. объясняет выбранную строку PIT join;
5. изменяет TTL, время или entity key по заданию преподавателя;
6. интерпретирует coverage и feature age;
7. объясняет различие historical и online retrieval.

27. Повторная сдача и стоп-факторы

Повторная сдача выполняется через переоткрытую issue, отдельный PR, новый audit и обновлённую evidence card. История первой попытки сохраняется.

Стоп-факторы:

- latest join выдан за point-in-time корректный;

- available time отсутствует или игнорируется;
- target того же события используется как feature;
- TTL проверяется только текстом;
- результат имеет дубликаты request ID;
- online latest используется для historical training;
- Feast UI заменяет definitions и тест;
- общий audit выдан за личный;
- студент не может объяснить выбранную строку.

28. Критерии оценивания

Критерий	Баллы	Полное выполнение
Contract и временная семантика	2	entity и три времени определены однозначно
Point-in-time join	2	future, unavailable и stale rows исключены
Leakage audit и coverage	2	naive/correct доказательно сопоставлены
Тесты, CI и parity	2	сценарии воспроизводимы, contract связан с inference
Индивидуальный вклад и защита	2	личный audit, артефакт, review и объяснение

Максимум: 10 баллов. Рекомендуемый порог зачёта — 6 баллов при отсутствии стоп-факторов.

29. Чек-лист

- ☐ Команда состоит из 2–5 человек, роли ротированы.
- ☐ Entity key и timestamps описаны в contract.
- ☐ Request ID уникален.
- ☐ Naive output сохранён как отрицательный пример.
- ☐ Feature time не позже event time.
- ☐ Available time не позже event time.
- ☐ Feature age не превышает TTL.
- ☐ Coverage соответствует policy.
- ☐ Target не используется как feature.
- ☐ Historical и online retrieval разделены.
- ☐ Тесты и CI успешны.
- ☐ У каждого участника отдельный audit и evidence card.
- ☐ Итоговая версия отмечена tag lr13-v1.

30. Связь с ЛР14

В ЛР14 feature contract, owner, sources, ограничения, coverage и возможное влияние на сегменты переносятся в model card и data card. Использование внешних данных требует проверки лицензии и правил хранения.

31. Официальные источники

- Feast feature retrieval: https://docs.feast.dev/getting-started/concepts/feature-retrieval
- Feast feature view: https://docs.feast.dev/getting-started/concepts/feature-view
- Feast concepts: https://docs.feast.dev/getting-started/concepts
- Feast PyPI: https://pypi.org/project/feast/

ЛР14. Документация, безопасность и ответственное применение ML

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Продолжительность: 2 академических часа и самостоятельная работа
Предшествующая работа: ЛР13 «Point-in-time признаки и Feature Store»

1. Цель

Подготовить проверяемый комплект evidence ответственного учебного выпуска: model/data cards, групповые метрики, policy, threat model, security и license review; принять обоснованное решение.

2. Результаты обучения

Каждый студент должен уметь:

- связать документацию с точной версией системы;
- объяснить intended use и out-of-scope;
- рассчитать общие и групповые метрики;
- корректно обработать малую группу;
- интерпретировать gap без заявления о доказанной fairness;
- проследить угрозу от актива до контроля и residual risk;
- объяснить rotation секрета и allowlist логов;
- защитить собственное решение release/review/stop.

3. Командный результат

Команда предоставляет tag lr14-v1:

1. заполненные docs/model-card.md и docs/data-card.md;
2. version-controlled governance/responsible_ai_policy.json;
3. воспроизводимый reports/governance/lr14-report.json;
4. docs/security/threat-model.md и SECURITY.md;
5. реестр/процесс проверки лицензий;
6. responsible-use checklist;
7. минимум четыре теста и CI gate;
8. issue, PR, review и отдельная evidence card каждого участника.

4. Безопасная область работы

Используйте только синтетические данные курса. Не добавляйте реальные персональные, чувствительные или закрытые атрибуты. IsWeekend выбран как безопасный учебный сегмент для механики группового аудита и не считается защищённым признаком.

5. Подготовка

```
python -m pip install -e ".[dev]"
python -m pytest tests/test_governance.py -q
```

Проверьте, что файлы ЛР13 и предыдущие CI gates не повреждены.

6. Базовый запуск

```
python -m mlops_flight_delay.governance \
  --evaluation data/governance/evaluation.csv \
  --policy governance/responsible_ai_policy.json \
  --model-card docs/model-card.md \
  --output reports/governance/lr14-team-baseline.json
```

Ожидаются status pass, две опубликованные группы, полный model card и допустимый F1 gap.

7. Policy as code

Policy фиксирует версию, group column, classification threshold, minimum group rows, общий минимум F1, максимум F1 gap, обязательные разделы карточки и запрещённые поля логов. Изменение policy проходит отдельный PR; нельзя подгонять порог после просмотра результата без объяснения.

8. Model card

Замените учебные placeholders фактическими именами команды, registry version, commit SHA, периодом оценки и release decision. Каждое утверждение о качестве должно ссылаться на воспроизводимый отчёт.

9. Intended use

Укажите пользователя, момент прогноза, допустимое действие, данные и масштаб. Недопустимые сценарии перечислите явно. Формулировка «для аналитики» без границ не принимается.

10. Data card

Проверьте происхождение, состав, target, временное разбиение, качество, права, срок хранения, удаление и ограничения. Внешний набор без подтверждённых условий использования не допускается.

11. Evaluation dataset

Файл содержит request_id, y_true, prediction_probability и IsWeekend. Реальный проект должен фиксировать временное окно и связь с model version; учебный sample служит только для теста механики.

12. Общие метрики

Отчёт содержит rows, positive rate, predicted positive rate, precision, recall, F1 и ROC AUC. ROC AUC не вычисляется для группы с одним классом и сохраняется как null, а не придуманное значение.

13. Групповые метрики

Метрики считаются тем же кодом и threshold для каждой достаточной группы. Сохраняйте n: сравнение без объёма выборки не проходит защиту. Объясните различие base rate и predicted positive rate.

14. Minimum support

Создайте личный сценарий, в котором одна группа меньше policy minimum. Ожидается insufficient_sample, отсутствие метрик этой группы и общий status review. Не дублируйте строки ради прохождения gate.

15. F1 gap

Gap — разница максимального и минимального F1 среди достаточных групп. Он полезен как сигнал, но не является универсальной fairness-метрикой. Интерпретация включает n, base rate, другие ошибки и контекст использования.

16. Threshold experiment

Каждый участник либо команда выполняет контролируемое изменение threshold в отдельной policy-копии. Сравните precision, recall, predicted positive rate и gap. Не изменяйте baseline policy молча.

17. Completeness audit

Модуль извлекает H2/H3 headings и проверяет обязательные разделы model card. Удалите один heading в личной ветке: CI должен получить review/ненулевой exit. Наличие heading не заменяет review текста.

18. Threat model

Опишите активы, границы доверия и минимум пять угроз. Для каждой укажите последствие, preventive или detective control, владельца и остаточный риск. Хотя бы одна угроза относится к ML: poisoning, leakage, model substitution или неверная версия признаков.

19. Secret incident drill

Не создавая настоящего секрета, используйте строку DEMO_REVOKED_TOKEN в локальном fixture, который не commitится. Опишите порядок: остановить использование, отозвать/ротировать, оценить охват, исправить источник, очистить артефакты, задокументировать. Не помещайте значение в evidence.

20. Логи и privacy

Сверьте API/observability с prohibited_log_fields. Докажите, что логи содержат технические поля, но не raw features, token, password или personal data. Скриншот одного запроса недостаточен — нужен тест или поиск по воспроизводимому журналу.

21. Зависимости и лицензии

Получите список фактических runtime dependencies/SBOM разрешённым инструментом. Для выбранного компонента проверьте точную версию и лицензию по официальному источнику. Не копируйте случайную таблицу из Интернета и не делайте юридических заключений.

22. Release decision

Команда выбирает release, review или stop. Решение содержит дату, model/data version, policy, результат gates, известные ограничения, владельца, reviewer, mitigation и rollback trigger.

23. Автоматические тесты

Обязательны проверки:

1. общие и групповые метрики формируются;
2. F1 gap сравнивается с policy;
3. малая группа получает insufficient_sample;
4. отсутствующий heading обнаруживается;
5. неверные probabilities/labels отклоняются;
6. отчёт содержит предупреждение, что сегмент не доказывает fairness.

24. CI

Workflow запускает governance CLI после тестов и PIT-аудита, а JSON попадает в diagnostic artifacts. Неполный model card или нарушенный policy возвращает ненулевой exit. Любое исключение оформляется как явное решение преподавателя, а не continue-on-error.

25. Командные роли

Для команды 2–5 человек:

- владелец model card и release context;
- владелец data card/provenance/retention;
- владелец grouped metrics и policy;
- владелец threat model/security drill;

- владелец dependency/license evidence и CI.

В паре роли совмещаются. Роли должны отличаться от ЛР13; общий файл не отменяет личного артефакта.

26. Индивидуальная сдача участника команды

Командный результат: tag lr14-v1, cards, policy, audit, threat model, security/license evidence.

Каждый участник заполняет docs/evidence/lr14-evidence-card.md. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда и GitHub-логин
Лабораторная	ЛР14, tag и commit SHA
Роль	роль и отличие от ЛР13
Задача	собственная issue и критерий приёмки
Личный артефакт	section, policy, test, threat/control, license record или CI gate
Запуск	личный сценарий и отдельный output
Evidence	n, метрики, gap, completeness и статус
Решение	release/review/stop, альтернатива и residual risk
Review	ссылка на содержательный review другого участника
Использование ИИ	инструмент и самостоятельно проверенные утверждения

27. Минимальный допуск

1. Командная версия отмечена tag и commit SHA.
2. Есть собственная issue с наблюдаемым критерием.
3. Личный артефакт встроен в командную версию.
4. Изменение прошло PR и review другого участника.
5. Личный сценарий воспроизводится отдельной командой.
6. CI успешен либо исключение письменно принято преподавателем.
7. Студент объясняет ограничение своего evidence.

28. Индивидуальная защита

За 3–4 минуты участник:

1. показывает личный артефакт в tag;
2. воспроизводит свой negative/edge scenario;
3. интерпретирует n, метрики и status;
4. изменяет threshold или minimum support по заданию;
5. объясняет, почему результат не доказывает fairness;
6. прослеживает угрозу до контроля и residual risk;
7. называет условие остановки/rollback и владельца.

29. Повторная сдача и стоп-факторы

Повторная сдача: переоткрытая issue, отдельный PR, новый JSON и обновлённая evidence card. История первой попытки сохраняется.

Стоп-факторы:

- реальные чувствительные данные добавлены в репозиторий;

- секрет опубликован или показан в evidence;
- малые группы интерпретируются как надёжные;
- один gap назван доказательством fairness;
- model/data card не соответствует версии артефакта;
- лицензия придумана без проверки источника;
- security checklist заменяет threat model;
- CI gate отключён ради зелёного статуса;
- общий отчёт выдан за индивидуальный вклад;
- студент не может объяснить решение.

30. Критерии оценивания

Критерий	Баллы	Полное выполнение
Model/data cards и provenance	2	назначение, версии, права и ограничения проверяемы
Групповые метрики и policy	2	n, threshold, minimum support и gap корректны
Security и threat model	2	активы, границы, controls и residual risk связаны
Тесты, CI и release decision	2	gates воспроизводимы, решение имеет владельца
Индивидуальный вклад и защита	2	личный сценарий, evidence, review и объяснение

Максимум: 10 баллов. Порог зачёта — 6 баллов при отсутствии стоп-факторов.

31. Чек-лист

- ☐ Команда 2–5 человек, роли ротированы.
- ☐ Model card связан с model/data/commit versions.
- ☐ Intended use и out-of-scope конкретны.
- ☐ Data card содержит provenance, права и retention.
- ☐ Policy находится в Git и прошла review.
- ☐ Для каждой группы сохранён n.
- ☐ Малые группы подавляются.
- ☐ Ограничение fairness сформулировано явно.
- ☐ Threat model содержит residual risk.
- ☐ Секреты и raw features отсутствуют в evidence/логах.
- ☐ License record основан на официальном источнике.
- ☐ Тесты и CI успешны.
- ☐ У каждого участника отдельный output и evidence card.
- ☐ Итог отмечен tag lr14-v1.

32. Связь с ЛР15

В ЛР15 release constraints, SLO, security controls и допустимое назначение станут входами для нагрузочного теста и решения о масштабировании. Производительность без ограничений риска не является готовностью к выпуску.

33. Официальные источники

- NIST AI RMF: https://www.nist.gov/itl/ai-risk-management-framework
- NIST AI RMF Core: https://airc.nist.gov/airmf-resources/airmf/5-sec-core/
- Model Card Toolkit repository (архивирован): https://github.com/tensorflow/model-card-toolkit

- GitHub secret security: https://docs.github.com/en/code-security/concepts/secret-security
- scikit-learn model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html

ЛР15. Нагрузочный протокол, масштабирование и обзор LLMOps

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с индивидуальной сдачей
Продолжительность: 2 академических часа и самостоятельная работа
Предшествующая работа: ЛР14 «Документация, безопасность и Responsible ML»

1. Цель

Провести воспроизводимый steady/burst/recovery эксперимент, проверить SLO, локализовать bottleneck, сравнить capacity/cost варианты и оформить решение. Разобрать Kubernetes и LLMOps на обзорном уровне.

2. Результаты обучения

Каждый студент должен уметь:

- сформулировать workload model и условия воспроизводимости;
- получить и проверить сырые latency/status;
- рассчитать throughput, error rate, p50, p95 и max;
- объяснить minimum sample и coordinated omission;
- сопоставить latency с CPU, memory и saturation;
- провести recovery drill без разрушительных действий;
- обосновать keep/optimize/scale и оценку стоимости;
- объяснить HPA, probes, requests/limits и базовый LLMOps eval.

3. Командный результат

Команда предоставляет tag lr15-v1:

1. docs/capacity-plan.md с workload и SLO;
2. три повторных raw-профиля steady/burst;
3. recovery evidence с timestamps;
4. capacity/load_policy.json и итоговый capacity report;
5. графики/метрики ресурсов или обоснованное описание ограничения среды;
6. решение с bottleneck, альтернативой, стоимостью и rollback trigger;
7. review Kubernetes manifests и LLMOps eval cases;
8. тесты, CI и индивидуальные evidence cards.

4. Обязательный и обзорный контуры

Обязательны Docker Compose/API, Python load profile и capacity audit. Kubernetes cluster, metrics-server, внешний LLM API и платные вызовы не требуются. Manifests и eval cases проверяются как код и защищаются устно.

5. Подготовка

```
python -m pip install -e ".[dev]"
docker compose up --build --detach
curl http://127.0.0.1:8080/readyz
```

Зафиксируйте commit SHA, image ref, модель, CPU/RAM машины и отсутствие посторонней нагрузки.

6. Capacity policy

capacity/load_policy.json задаёт required scenarios, minimum requests, error-rate/p95 limits, recovery time, baseline/max replicas и учебную стоимость. Изменение policy после результата без отдельного обоснования считается подгонкой.

7. Warm-up

Скрипт выполняет пять запросов до измерения. Warm-up не смешивается со steady. Если модель загружается лениво или использует cache, увеличьте warm-up и зафиксируйте правило до сравнения вариантов.

8. Личный базовый запуск

```
python scripts/load_profile.py \
  --base-url http://127.0.0.1:8080 \
  --student-id student-01 \
  --requests-per-stage 40 \
  --steady-concurrency 2 \
  --burst-concurrency 10 \
  --output reports/capacity/lr15-student-01-run1-raw.json
```

Повторите три раза с теми же параметрами и уникальными output names.

9. Workload model

Учебный генератор использует закрытую модель с ThreadPoolExecutor. Укажите число workers, запросов, payload, timeout и среду. На защите объясните, почему медленный ответ уменьшает фактический arrival rate и как это связано с coordinated omission.

10. Сырые данные

Raw JSON хранит одну latency на запрос, successes/errors, wall duration и concurrency. Нельзя оставлять только скриншот terminal или готовый p95: преподаватель должен пересчитать percentile.

11. Capacity audit

```
python -m mlops_flight_delay.capacity \
  --results reports/capacity/lr15-student-01-run1-raw.json \
  --policy capacity/load_policy.json \
  --output reports/capacity/lr15-student-01-run1.json
```

--enforce разрешён для утверждённого release gate; экспериментальный fail сохраняется как evidence.

12. Percentiles

Проект использует nearest-rank: ceil(q*n). Сравните p50, p95 и max. Покажите одну строку ручного расчёта или тест. Не округляйте входы до вычисления percentile.

13. Throughput

Throughput равен requests / wall duration. Он считается вместе с latency: рост throughput ценой timeouts не является улучшением. Укажите фактическую concurrency и проверьте приблизительное соответствие закону Литтла.

14. Error classification

Отделите timeout/connection/5xx от намеренных 4xx. В обязательном load profile payload валиден. Если добавлен negative scenario, его 422 хранится в отдельном результате и не смешивается с capacity error rate.

15. Наблюдение ресурсов

Во время steady и burst сохраните CPU, memory и request rate из Prometheus/Grafana либо docker stats. Укажите timestamps, чтобы сопоставить ресурс и p95. Скриншот без периода и параметров нагрузки недостаточен.

16. Bottleneck hypothesis

Выберите один наблюдаемый bottleneck или докажите, что он не достигнут. Рассмотрите CPU, memory, client, I/O, worker/connection limit и загрузку модели. Гипотеза должна предсказывать эффект конкретного изменения.

17. Контролируемый эксперимент

Измените один фактор: concurrency, workers, CPU limit или replicas в локальном Compose-эквиваленте. Не меняйте одновременно код модели и инфраструктуру. Повторите тот же профиль и сравните таблицу.

18. Recovery drill

В согласованное окно выполните безопасный docker compose restart api. Измерьте время от команды до трёх последовательных 200 на /readyz; сохраните timestamps и ошибки. Не удаляйте volumes и не используйте реальные production-сервисы.

19. Критерий устойчивой готовности

Один ответ 200 не завершает recovery. Требуются три последовательные успешные readiness checks. Если сервис флапает, зафиксируйте последовательность, logs и причину вместо усреднения.

20. Решение о capacity

pass предлагает keep_baseline. review предлагает investigate_then_scale, а не автоматическое увеличение replicas. Команда принимает итоговое keep/optimize/scale/limit/redesign с владельцем.

21. Стоимость

Сравните baseline и candidate в условных единицах replica-hour. Укажите, что не учтены registry, load balancer, storage, egress и сопровождение. Не выдавайте учебную оценку за тариф провайдера.

22. Kubernetes Deployment review

Проверьте immutable image, rolling strategy, non-root, read-only filesystem, requests/limits, startup/readiness/liveness и termination grace. Каждая probe должна иметь отдельную семантику.

23. HPA review

k8s/hpa.yaml использует autoscaling/v2, 2–6 replicas, CPU target и scale-down stabilization. Объясните необходимость resource requests и metrics pipeline. CPU HPA не гарантирует p95 и может не помочь при I/O bottleneck.

24. PDB review

k8s/pdb.yaml ограничивает добровольные disruptions до minAvailable: 1. Объясните, какие crash и ошибки он не предотвращает. Kubernetes manifests не применяются к чужому cluster.

25. LLMOps mapping

Сопоставьте классический артефакт с LLMOps:

MLOps	LLMOps
model/data version	provider/model, prompt, retrieval corpus
prediction log	trace шагов и tool calls

MLOps	LLMOps
test set	eval cases и critical slices
metric gate	deterministic/model/human scorers
latency/cost	TTFT, total latency, tokens и стоимость ответа
input security	injection, data leakage и tool boundaries

26. Eval cases

llmops/eval_cases.json содержит format, refusal, injection и grounding cases. Каждый участник добавляет или рецензирует один безопасный case и указывает тип scorer. Реальный API не вызывается, секрет не создаётся.

27. MLflow GenAI обзор

Покажите в документации назначение tracing и mlflow.genai.evaluate(). Не смешивайте его с mlflow.models.evaluate() классического ML. Укажите, какие поля trace нельзя сохранять безусловно.

28. Автоматические тесты

Обязательны проверки:

1. nearest-rank percentile детерминирован;
2. baseline проходит policy;
3. высокий burst p95 приводит к review;
4. отсутствующий scenario отклоняется;
5. decision не обещает автоматическое масштабирование;
6. JSON/YAML parseable и версии согласованы.

29. CI

CI анализирует синтетический sample и сохраняет reports/capacity/lr15-ci.json. Live load test не запускается на shared runner: его шумный результат не является стабильным unit gate. Собственные raw прогоны прикладываются как artifacts к PR.

30. Командные роли

Для команды 2–5 человек:

- владелец workload/protocol и генератора;
- владелец metrics/SLO и capacity audit;
- владелец resource observation и bottleneck experiment;
- владелец recovery/Kubernetes review;
- владелец cost decision и LLMOps eval review.

В паре роли совмещаются; координатор без измерения, review или технического артефакта не засчитывается.

31. Индивидуальная сдача участника команды

Командный результат: tag lr15-v1, протокол, raw runs, capacity report, recovery и decision record.

Каждый заполняет docs/evidence/lr15-evidence-card.md; совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда и GitHub-логин
Лабораторная	ЛР15, tag и commit SHA

Поле	Что указать
Роль	роль и отличие от ЛР14
Задача	собственная issue с критерием приёмки
Личный артефакт	профиль, тест, SLO, recovery, manifest review или eval case
Среда	CPU/RAM, image/model version, команда запуска
Evidence	requests, concurrency, throughput, p50/p95/max, errors, recovery, resources
Решение	bottleneck, альтернатива, cost/risk и rollback trigger
Review	содержательное замечание и принятое решение
Использование ИИ	инструмент и самостоятельно проверенные утверждения

32. Минимальный допуск

1. Командная версия зафиксирована tag и commit SHA.
2. Участнику назначена собственная issue.
3. Есть личный raw output либо иной проверяемый технический артефакт.
4. PR проверен другим участником.
5. Параметры и среда воспроизводимы.
6. CI успешен либо исключение письменно принято преподавателем.
7. Студент объясняет ограничение метода.

33. Индивидуальная защита

За 3–4 минуты участник:

1. показывает свой артефакт в tag;
2. пересчитывает p95 или throughput;
3. интерпретирует resource evidence;
4. меняет concurrency/SLO по заданию;
5. объясняет bottleneck и альтернативу scale;
6. читает один фрагмент HPA/probe;
7. классифицирует LLMOps eval case и его риск.

34. Повторная сдача и стоп-факторы

Повторная сдача выполняется через переоткрытую issue, отдельный PR, новый raw run и evidence card. История первой попытки сохраняется.

Стоп-факторы:

- изображение графика заменяет raw данные;
- сравниваются разные версии/среды без указания;
- 4xx смешаны с server capacity errors;
- p95 рассчитан без фиксированного метода;
- один аномальный run объявлен доказательством;
- replicas увеличены без bottleneck evidence;
- стоимость представлена как точный тариф;
- Kubernetes cluster или внешний LLM ошибочно сделан обязательным;
- секрет/API key попал в Git;
- общий run выдан за личный;

- студент не объясняет свой вывод.

35. Критерии оценивания

Критерий	Баллы	Полное выполнение
Протокол и воспроизводимость	2	workload, версии, среда и raw data зафиксированы
Метрики и SLO	2	throughput, errors, p50/p95/max рассчитаны корректно
Bottleneck и recovery	2	ресурсы сопоставлены, recovery доказан timestamps
Capacity/cost, Kubernetes, LLMOps	2	решение и ограничения объяснены без магии
Индивидуальный вклад и защита	2	личный артефакт, review и диагностика воспроизводимы

Максимум: 10 баллов. Порог зачёта — 6 баллов при отсутствии стоп-факторов.

36. Чек-лист

- ☐ Команда 2–5 человек, роли ротированы.
- ☐ Commit, image, model и policy versions записаны.
- ☐ Warm-up отделён от измерения.
- ☐ Есть три повторных steady/burst runs.
- ☐ Сырые latency сохранены.
- ☐ p50/p95/max и throughput пересчитываются.
- ☐ Ошибки классифицированы.
- ☐ Ресурсы связаны с тем же периодом.
- ☐ Recovery требует трёх readiness successes.
- ☐ Bottleneck подтверждён либо ограничение признано.
- ☐ Cost assumptions явны.
- ☐ Kubernetes остаётся обзором/демо.
- ☐ LLMOps eval не требует секрета.
- ☐ CI и индивидуальные cards готовы.
- ☐ Итог отмечен tag lr15-v1.

37. Связь с ЛР16

В ЛР16 выбранный capacity decision, recovery evidence, model/data versions, image ref и governance status войдут в единый release manifest. Команда проведёт итоговую end-to-end демонстрацию и индивидуальную защиту сквозного проекта.

38. Официальные источники

- Kubernetes HPA: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/
- Kubernetes resources: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
- Kubernetes probes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-probes/
- MLflow GenAI tracing: https://mlflow.org/docs/latest/genai/getting-started/tracing
- MLflow GenAI evaluation API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.genai.html

ЛР16. Интеграция, release manifest и защита проекта

Версия: 2026–2027 учебный год
Формат: команда 2–5 человек с обязательной индивидуальной защитой
Продолжительность: 2 академических часа и самостоятельная подготовка
Предшествующая работа: ЛР15 «Нагрузочный протокол, масштабирование и обзор LLMOps»

1. Цель

Собрать проверяемый итоговый выпуск сквозного ML-проекта, доказать связь кода, данных, модели, image, API и gates, выполнить end-to-end демонстрацию и индивидуально защитить инженерные решения.

2. Результаты обучения

Каждый студент должен уметь:

- прочитать и проверить release manifest;
- проследить commit → CI → image и data → run → model version;
- отличить alias champion от exact version;
- проверить SHA-256 evidence;
- объяснить status gate и управляемое исключение;
- выполнить диагностический шаг или rollback check;
- сформулировать ограничение и остаточный риск;
- доказать личный вклад в общей системе.

3. Командный результат

Команда предоставляет release tag course-final-v1:

1. работающий end-to-end репозиторий;
2. reports/releases/lr16-manifest.json со статусом ready или ready_with_exception;
3. успешный CI run и immutable image identity;
4. exact MLflow model version и проверенный alias champion;
5. актуальные model/data cards, security, monitoring и capacity evidence;
6. проверенный rollback и post-rollback smoke;
7. итоговый отчёт/README и сценарий демонстрации;
8. отдельная evidence card каждого участника.

4. Что не требуется повторять

Не нужно заново писать все pipeline. Используйте принятые артефакты ЛР1–ЛР15, исправляя только несогласованности. Финальная задача — связать identities и доказательства, а не создать параллельную копию проекта для защиты.

5. Freeze и preparation branch

Создайте issue финальной интеграции и короткоживущую branch. За 24 часа до защиты зафиксируйте content freeze: новые изменения проходят отдельный PR и повторяют затронутые gates. Не переписывайте историю после формирования manifest.

6. Инвентаризация identities

Зафиксируйте:

- full source commit SHA и release tag;
- data/feature contract versions и dataset identity;

- MLflow run ID, model name, version, alias и resolved URI;
- image digest либо full-commit tag;
- API/OpenAPI version;
- policy versions и environment.

Любое latest, «текущий» или «последний run» заменяется точной identity.

7. Копирование example spec

```
cp release/release_spec.example.json release/release_spec.json
```

Заполните реальными значениями команды. release_spec.example.json намеренно описывает draft и не может быть выдан за готовый выпуск.

8. Код и image

source_commit содержит 40 hex-символов. image.ref использует registry digest либо tag, равный этому SHA. SemVer можно добавить как удобный alias, но он не заменяет immutable identity.

9. Данные и признаки

Проверьте соответствие data/contract.json, DVC/источника, хронологического split и PIT audit. Manifest ссылается на evidence, а data/model cards объясняют происхождение, права и ограничения.

10. Модель

Разрешите alias champion и сохраните exact version. resolved_uri должен заканчиваться номером этой версии. Покажите MLflow run, tags source_commit, data_sha256 и validation status.

11. API

Экспортируйте OpenAPI из release commit и выполните contract tests. Версия в schema должна совпасть с package/API version. Breaking change требует явной новой версии и плана миграции клиента.

12. Обязательные gates

Release policy требует:

1. data contract;
2. point-in-time features;
3. monitoring decision;
4. responsible AI;
5. capacity;
6. model card;
7. security.

Добавьте тесты/CI/API evidence по правилам команды, но не удаляйте обязательные gates.

13. Статусы gates

pass означает выполненную policy и доказательство. accepted_with_investigation допускается только с owner, expires_at и reason. fail, missing evidence или истёкшее исключение блокируют release.

14. Сборка manifest

```
python -m mlops_flight_delay.release build \
  --spec release/release_spec.json \
  --policy release/release_policy.json \
```

```
  --output reports/releases/lr16-manifest.json \
  --require-ready
```

Builder проверяет identities, наличие gates/evidence и вычисляет SHA-256 каждого файла.

15. Проверка manifest

```
python -m mlops_flight_delay.release verify \
  --manifest reports/releases/lr16-manifest.json \
  --policy release/release_policy.json
```

После изменения любого evidence manifest перестаёт проходить verify и должен быть пересобран с повторным review.

16. CI identity

Укажите URL успешного run для release commit. Локальный успешный запуск не заменяет командный CI. Если GitHub недоступен по независимой причине, преподаватель письменно принимает offline evidence; самостоятельно менять status на success запрещено.

17. Independent approval

prepared_by и approved_by различаются. Reviewer проверяет exact identities, hashes, исключения и rollback, а не только нажимает Approve. Автор может исправить замечание, но не подтверждает его сам.

18. Ready и ready_with_exception

Без CI и approval manifest остаётся draft. Все gates pass дают ready; принятое временное исключение — ready_with_exception. Команда объясняет влияние exception и план его закрытия.

19. Container/API smoke

Запустите release image и сохраните:

```
curl http://127.0.0.1:8080/healthz
curl http://127.0.0.1:8080/readyz
curl http://127.0.0.1:8080/metrics
python scripts/api_contract_smoke.py --base-url http://127.0.0.1:8080
```

Покажите valid/invalid request, request ID, resolved model identity и отсутствие raw features в log.

20. Post-release checks

Проверьте readiness, error rate, latency и model identity после запуска. Deployment command без фактической проверки сервиса не считается успешным release.

21. Rollback preparation

До drill зафиксируйте предыдущие exact model/image identities, причину/trigger, approver и smoke criteria. Если совместимая предыдущая пара отсутствует, release не готов к защите.

22. Rollback drill

Переместите alias champion на проверенную предыдущую model version и верните предыдущий immutable image либо выполните согласованный локальный эквивалент. Не используйте production и не удаляйте данные. Сохраните from/to identities, timestamps и результат.

23. Проверка после rollback

Выполните smoke prediction, /readyz, contract check и проверку resolved model. Один успешный deployment status не доказывает работоспособность. После drill верните согласованное состояние новым управляемым действием и зафиксируйте его.

24. Artifact attestation — расширение

Если репозиторий/тариф поддерживает attestations, сформируйте provenance для release artifact и проверьте gh attestation verify. Это дополнительный балл evidence, но не замена tests, security, model quality и release approval.

25. Итоговый README/отчёт

Он содержит задачу, архитектуру, quick start, contracts, pipeline, monitoring, ограничения, responsible use, release/rollback, ссылки на manifest и распределение вкладов. Приватный СКИФ не используется вместо самодостаточного описания.

26. Сценарий командной демонстрации

Используйте docs/final-demo-script.md. Продолжительность 10–12 минут. Назначьте ведущего каждого шага и запасной вариант на случай сетевой недоступности. Запись экрана не заменяет live explanation.

27. Роли команды

Для 2–5 человек:

- release/data/model identity owner;
- API/container/integration owner;
- CI/evidence/manifest owner;
- observability/governance/capacity owner;
- rollback/security/demo reviewer.

В паре роли совмещаются. Каждый участник выполняет технический или проверочный шаг; роль только ведущего презентации не является индивидуальным вкладом.

28. Индивидуальная сдача участника команды

Командный результат: course-final-v1, готовый manifest, CI, demo и rollback evidence.

Каждый заполняет docs/evidence/lr16-evidence-card.md; совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Release	tag, manifest SHA и source commit
Роль	роль и отличие от ЛР15
Задача	собственная issue с критерием приёмки
Личный артефакт	integration fix, gate, evidence, demo step, rollback или review
Вклад	конкретный diff/решение и его связь с release
Команда	индивидуальный воспроизводимый запуск и output
Решение	альтернатива, критерий, риск и ограничение
Диагностика	действие преподавателя и результат
Review	замечание другого участника и реакция команды
Использование ИИ	инструмент и самостоятельно проверенные утверждения

29. Минимальный допуск

1. Release tag и full commit SHA зафиксированы.
2. Manifest имеет ready или ready_with_exception.
3. CI run успешен либо есть письменное решение преподавателя.
4. У студента собственная issue и встроенный личный артефакт.

5. PR прошёл review другого участника.
6. Evidence card содержит рабочие identities/ссылки.
7. Rollback подготовлен и безопасен.
8. Студент способен выполнить диагностическую операцию.

30. Индивидуальная защита

За 3–4 минуты студент:

1. показывает личный артефакт в release tag;
2. прослеживает его до manifest/gate;
3. выполняет одну операцию: verify hash, resolve alias, contract smoke, broken-copy gate, readiness check, интерпретация alert или post-rollback smoke;
4. объясняет решение, альтернативу и остаточный риск;
5. отвечает на вопрос по соседнему связанному компоненту.

31. Набор диагностических заданий преподавателя

- заменить image ref на latest в копии spec и объяснить отказ;
- изменить evidence и запустить verify;
- показать exact version за alias;
- определить, почему manifest остаётся draft;
- найти owner/expiry исключения;
- сопоставить API version с image;
- выбрать rollback или roll-forward;
- найти поле, которое нельзя логировать;
- объяснить, почему attestation не доказывает качество модели.

32. Стоп-факторы

- release основан на latest или неизвестном commit;
- alias выдан без exact model version;
- manifest вручную объявлен ready без CI/approval;
- pass не имеет evidence/hash;
- исключение бессрочно или без владельца;
- author и approver совпадают;
- rollback не содержит точных identities;
- секреты/персональные данные опубликованы;
- общая демонстрация выдана за личную защиту;
- участник не объясняет собственный артефакт.

33. Повторная сдача

Переоткройте issue, исправьте отдельным PR, повторите затронутые gates, пересоберите manifest и обновите evidence card. Создайте новый release tag; старый не перемещается и первая попытка не удаляется. Повторно защищается исправленная и связанная с ней часть.

34. Критерии оценивания

Критерий	Баллы	Полное выполнение
Release identities и manifest	2	code/data/model/image/API связаны точно

Критерий	Баллы	Полное выполнение
Gates, CI и approvals	2	evidence проверяемо, исключения управляются
End-to-end демонстрация	2	запуск, API, monitoring и ограничения показаны
Rollback и post-check	2	exact previous state восстановлено и проверено
Индивидуальный вклад и защита	2	личный артефакт, review и диагностика доказаны

Максимум: 10 баллов. Порог зачёта — 6 баллов при отсутствии стоп-факторов. Командная и индивидуальная оценки могут различаться.

35. Чек-лист

- ☐ Команда 2–5 человек, у каждого техническая/проверочная роль.
- ☐ Release tag не перемещается.
- ☐ Full source commit записан.
- ☐ Data/feature identities согласованы.
- ☐ MLflow run, exact version и alias разрешены.
- ☐ Image immutable и соответствует commit.
- ☐ API/OpenAPI version совпадает.
- ☐ Обязательные gates присутствуют.
- ☐ Evidence files существуют и hashes проверяются.
- ☐ Исключения имеют owner/expiry/reason.
- ☐ CI успешен и approval независим.
- ☐ Manifest не draft.
- ☐ Container/API smoke выполнен.
- ☐ Rollback и post-check воспроизводимы.
- ☐ Cards каждого участника уникальны.
- ☐ Итог отмечен course-final-v1.

36. После защиты

Команда сохраняет manifest, отчёт, cards и ссылки на CI согласно правилам курса. Секреты и реальные данные не архивируются. Незакрытые исключения передаются владельцу; учебный сервис останавливается, если его дальнейшая работа не согласована.

37. Официальные источники

- GitHub artifact attestations: https://docs.github.com/en/actions/concepts/security/artifact-attestations
- GitHub deployments/environments: https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments
- GitHub Actions security: https://docs.github.com/en/actions/how-tos/secure-your-work
- MLflow Model Registry workflow: https://mlflow.org/docs/latest/ml/model-registry/workflow/

Приложение А. Сценарий итоговой демонстрации

1. Показать release tag, commit и reports/releases/lr16-manifest.json.
2. Воспроизвести окружение и выполнить data/feature validation.
3. Показать MLflow exact version и разрешение alias champion.
4. Запустить контейнер по immutable image ref; проверить /healthz, /readyz, /metrics.

5. Выполнить валидный и невалидный API-запрос, показать request ID и безопасный лог.
6. Показать monitoring/governance/capacity решения и известное ограничение.
7. Выполнить контролируемый rollback на точную предыдущую model/image identity.
8. Подтвердить smoke/readiness после rollback и вернуть согласованное состояние.

Командная демонстрация рассчитана на 10–12 минут. Затем каждый участник за 3–4 минуты показывает личный артефакт и выполняет диагностическую операцию преподавателя.

Приложение Б. Release checklist

- ☐ Release tag и source commit неизменяемы и совпадают с manifest.
- ☐ Data/feature contracts, dataset identity и point-in-time audit указаны.
- ☐ MLflow run, exact model version, alias champion и resolved URI согласованы.
- ☐ Image использует digest или commit-SHA tag; latest отсутствует.
- ☐ OpenAPI version и результат contract tests зафиксированы.
- ☐ CI run успешен; автор и release reviewer различаются.
- ☐ Drift/quality decision, governance и capacity evidence имеют SHA-256.
- ☐ Исключение имеет причину, владельца и срок пересмотра.
- ☐ Security, licenses, model/data cards и ограничения актуальны.
- ☐ Rollback указывает точные предыдущие model/image identities.
- ☐ Демонстрация и индивидуальные evidence cards воспроизводимы.
- ☐ После rollback выполнены smoke/readiness checks и записано решение.

GitHub artifact attestation является дополнительным доказательством происхождения build, но не доказывает безопасность или качество артефакта. Если команда её использует, сохраняются команда и результат gh attestation verify.

Приложение В. Responsible use checklist

- ☐ Назначение, пользователи и недопустимые сценарии описаны.
- ☐ Data card содержит происхождение, права, ограничения и срок хранения.
- ☐ Model card соответствует фактической версии модели и порогу.
- ☐ Группы выбраны по обоснованной гипотезе риска; малые выборки не интерпретируются.
- ☐ Средняя метрика не скрывает критический сегмент или стоимость ошибки.
- ☐ Назначены владелец, reviewer, условие остановки и rollback.
- ☐ Секреты/персональные данные отсутствуют в Git, логах и evidence.
- ☐ Threat model и лицензии обновлены.
- ☐ Ограничения сообщены человеку, принимающему решение.
- ☐ Каждое существенное утверждение подтверждено воспроизводимым артефактом.